影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 12 章 視訊與動態影像處理

本章學習目標

- 說明視訊的本質(連續影格)與時間維度為影像處理帶來的新機會與新挑戰。
- 以 OpenCV 讀取、處理、寫出視訊與即時攝影機串流，理解影格率與編解碼器。
- 運用影格差分與背景相減完成移動偵測，並以型態學清理前景遮罩。
- 闡述光流的概念，區分稀疏光流(Lucas-Kanade)與稠密光流(Farnebäck)。
- 實作物件追蹤，區分「偵測」與「追蹤」的差異，並串接第 8、9、10、16 章。
- 認識視訊穩定、動作辨識與時序深度學習，理解時間維度在機器人視覺中的角色。

叫獸開講|一場關於奔馬的爭論，如何意外催生了電影

故事要從一個看似無聊的問題說起：一匹馬全速奔跑時，會不會有那麼一瞬間，四隻蹄子同時離地?在 1870 年代，這個問題無法回答——馬腿動得太快，人眼根本看不清。傳統繪畫總把奔馬畫成「飛躍式」：四腿向前後極力伸展、如同騰空飛翔。但這真的是馬奔跑的樣子嗎?沒有人能證明。

加州的鐵路大亨、前州長 Leland Stanford(後來史丹佛大學的創辦人)對這個問題著了迷。1872 年，他請來英國攝影師 Eadweard Muybridge 來解決它。流傳最廣的版本說，Stanford 為此下了兩萬五千美元的賭注——不過要誠實說，不少歷史學家認為這個「賭局」缺乏一手史料佐證，很可能是後人加油添醋的傳說。無論賭注是真是假，可以確定的是：這是一項需要當時最頂尖攝影技術的艱鉅挑

戰，而 Stanford 願意持續投入重金。

經過六年的技術摸索(期間 Muybridge 還一度因槍殺妻子的情夫而中斷工作、受審),1878 年 6 月，他終於在史丹佛的帕羅奧圖農場架設起一整排相機——沿著跑道平行排列，相機的快門連接絆線，當名為「Sallie Gardner」的母馬奔馳而過、依序碰斷絆線，便觸發一台台相機，以大約二十五分之一秒的間隔連續拍下一整組動作。照片沖洗出來，答案揭曉：確實有一個瞬間，馬的四蹄全部離地——但不是傳統繪畫想像的伸展飛躍姿，而是四腿收攏於腹下的那一刻。從此，「飛躍式奔馬」的錯誤畫法在藝術界銷聲匿跡。

但這個故事真正的遺產遠不止於此。當 Muybridge 把這一系列靜態照片依序快速播放，觀眾看到的竟是一匹「活生生奔跑的馬」——史上第一次，連續的靜態影像創造出了「動態」的錯覺。這組名為《奔跑的馬》(Sallie Gardner at a Gallop)的照片，被公認為電影的源頭。一場關於馬蹄的爭論，意外開啟了整個電影工業。而它揭示的原理，正是本章的基石：視訊，本質上就是一疊「快速翻頁的照片」;而所謂的「動態」，是我們的視覺系統把一連串靜態影格腦補連接起來的結果。

這也帶出了視訊處理的核心洞見。前十一章，我們把每一張影像當成孤立的個體來處理;但視訊多了一個維度——時間。相鄰的影格之間有著極強的關聯(這一影格和上一影格通常只差一點點)，這個「時間上的連續性」既是新的資訊來源(可以偵測移動、追蹤物件、估計速度)，也是新的挑戰(要即時處理、要在影格與影格之間維持一致)。學會利用時間這個維度，是本章的目標，也是機器人「看懂動態世界」的關鍵。

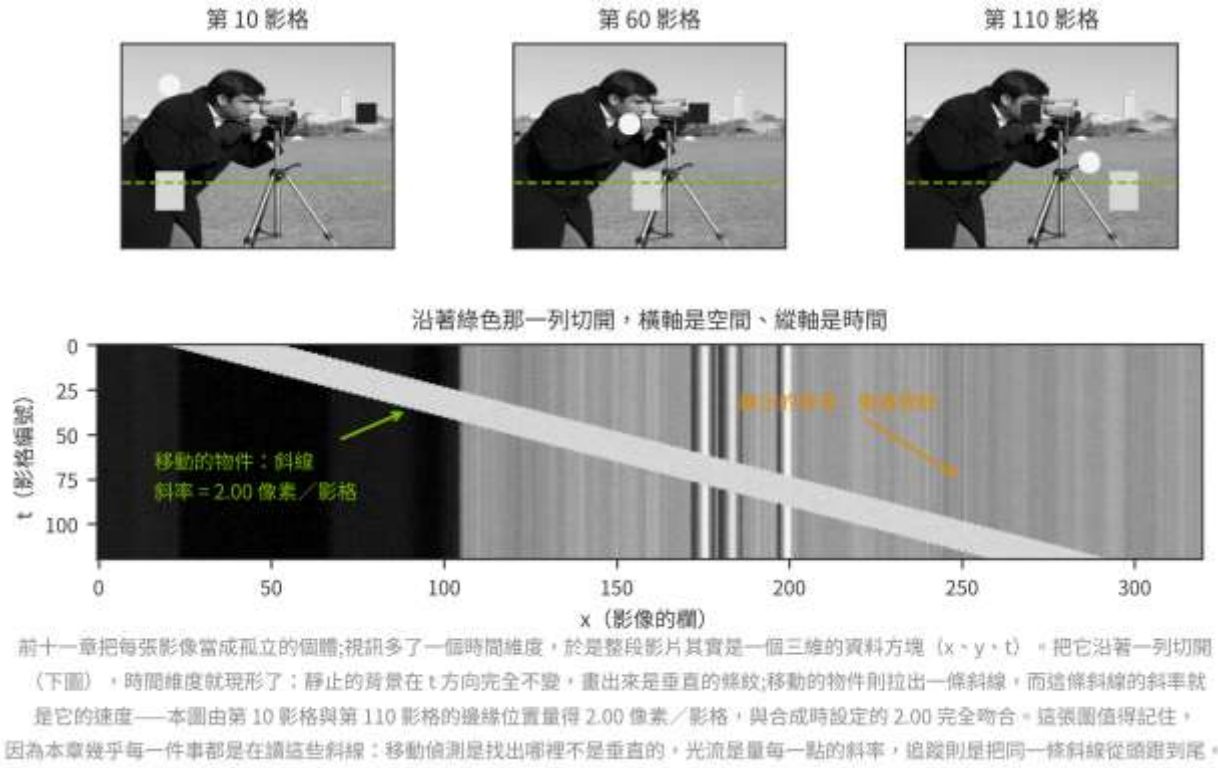
思考題：Muybridge 用「一排相機依序觸發」來把時間切成一格一格。今天的手機用單一感光元件、以固定影格率連續讀取來達成同樣的事。想一想：如果影格率不夠高(取樣時間的間隔太大)，拍攝高速旋轉的電風扇或車輪，會看到什麼奇怪的現象?這與第 3、6 章的哪個概念有關?

12.1 視訊的本質：多了一個時間維度

12.1.1 從影像到視訊

一張影像是二維的(寬、高)，而視訊多了第三個維度——時間。視訊是由一連串影格(frame)以固定的時間間隔排列而成，每一個影格都是一張完整的影像。播放的速度稱為影格率(frame rate)，單位是每秒影格數(fps)：電影通常是 24 fps，電視是 25 或 30 fps，而流暢的遊戲與高速攝影可達 60 fps 以上。當影格切換得夠快(一般認為超過每秒十餘格)，人類的視覺暫留就會把離散的影格「腦補」成連續的動態——這正是叫獸開講中 Muybridge 的發現，也是所有動態影像的生理基礎。

這個「時間軸」的加入，徹底改變了影像處理的格局。它同時帶來了機會與挑戰，理解這個雙面性，是掌握本章的關鍵。



前十一章把每張影像當成孤立的個體;視訊多了一個時間維度，於是整段影片其實是一個三維的資料方塊（x、y、t），把它沿著一列切開（下圖），時間維度就現形了：靜止的背景在 t 方向完全不變，畫出來是垂直的條紋;移動的物件則拉出一條斜線，而這條斜線的斜率就是它的速度——本圖由第 10 影格與第 110 影格的邊緣位置量得 2.00 像素／影格，與合成時設定的 2.00 完全吻合。這張圖值得記住，因為本章幾乎每一件事都是在讀這些斜線：移動偵測是找出哪裡不是垂直的，光流是量每一點的斜率，追蹤則是把同一條斜線從頭跟到尾。

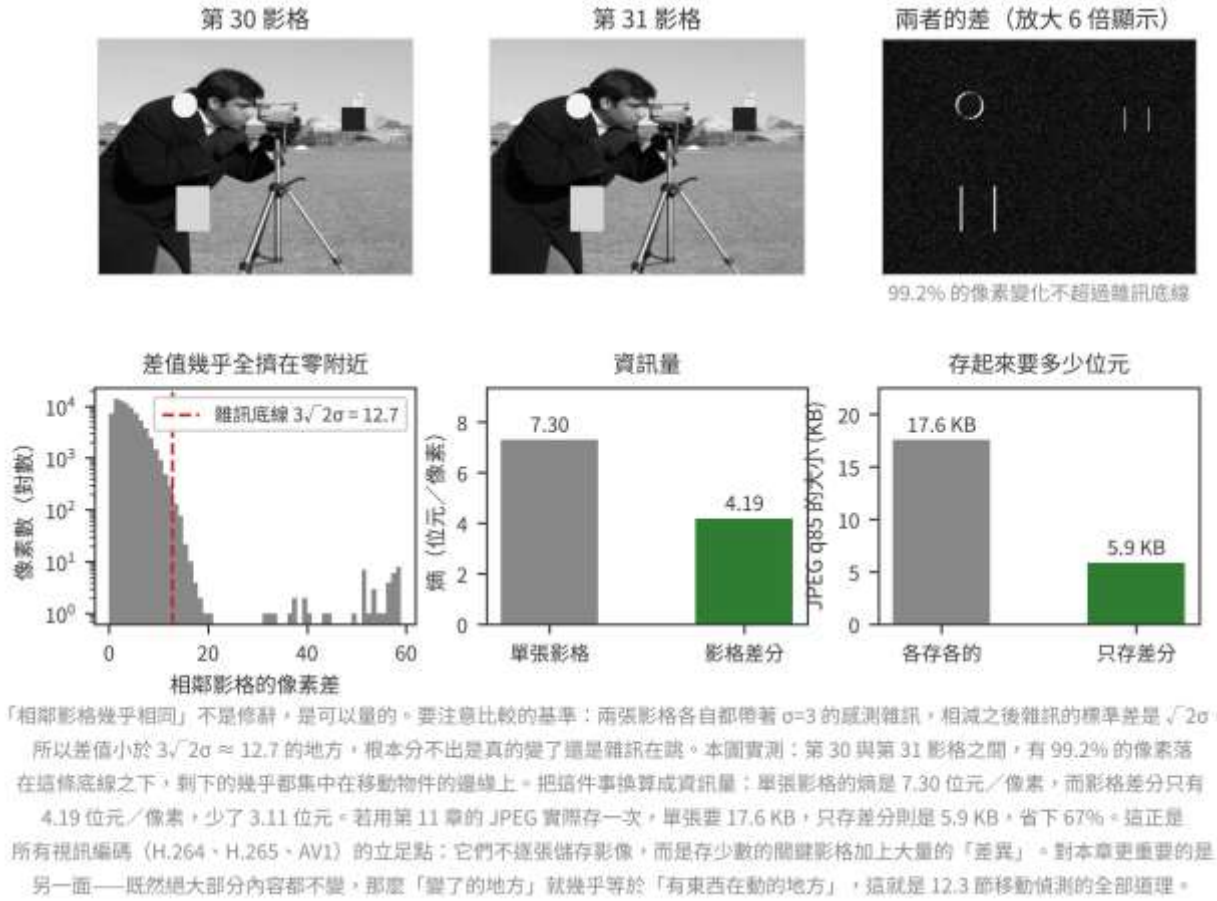
圖 12-1　把視訊切開來看：靜止是垂直條紋，移動是斜線

12.1.2 時間維度帶來的機會

相鄰影格之間有極強的時間關聯性——這一影格和上一影格，通常絕大部分內容

都相同，只有移動的物件或變化的區域不同。這個「幾乎不變」的特性，是一座資訊金礦：

- 偵測移動：把兩影格相減，不變的背景相消為零，只剩下移動的物件浮現(12.3 節)。單張影像永遠做不到這件事——「移動」本身就是一個跨越時間的概念。
- 追蹤物件：知道一個物件在這一影格的位置，由於下一影格它不會跳太遠，可以有效率地在附近找到它、串起它的軌跡(12.5 節)。
- 估計運動與速度：量測物件在連續影格間的位移，除以時間，就得到速度——這是機器人閃避障礙、預測行人動向的基礎(12.4 節光流)。
- 提升品質：多影格之間的冗餘可用於去雜訊(第 5 章夜景模式的多影格平均)、超解析度、視訊穩定。



「相鄰影格幾乎相同」不是修辭，是可以量的。要注意比較的基準：兩張影格各自都帶著 σ=3 的感測雜訊，相減之後雜訊的標準差是 √2σ，所以差值小於 3√2σ ≈ 12.7 的地方，根本分不出是真的變了還是雜訊在跳。本圖實測：第 30 與第 31 影格之間，有 99.2% 的像素落在這條底線之下，剩下的幾乎都集中在移動物件的邊線上。把這件事換算成資訊量：單張影格的熵是 7.30 位元／像素，而影格差分只有 4.19 位元／像素，少了 3.11 位元。若用第 11 章的 JPEG 實際存一次，單張要 17.6 KB，只存差分則是 5.9 KB，省下 67%。這正是所有視訊編碼（H.264、H.265、AV1）的立足點：它們不逐張儲存影像，而是存少數的關鍵影格加上大量的「差異」。對本章更重要的是另一面——既然絕大部分內容都不變，那麼「變了的地方」就幾乎等於「有東西在動的地方」，這就是 12.3 節移動偵測的全部道理。

圖 12-2　時間冗餘有多大：相鄰影格的差、熵與位元數

12.1.3 時間維度帶來的挑戰

但時間維度也帶來三個新挑戰：其一是即時性——視訊以每秒數十影格的速度湧入，處理演算法必須跟得上，否則就會掉影格或延遲，這對第 17 章的邊緣部署是嚴苛的考驗(一個在單張影像上跑得很好、但每一影格要一秒的演算法，對 30 fps 的視訊毫無用處)。其二是時間一致性——處理結果在影格與影格之間必須穩定，若每一影格獨立處理，偵測框可能會抖動、閃爍，追蹤編號可能會跳號。其三是資料量——視訊的資料量是影像的數十倍，儲存與傳輸都仰賴第 11 章的壓縮技術(視訊編碼更進一步利用了「影格間冗餘」，只記錄影格與影格之間的差異)。

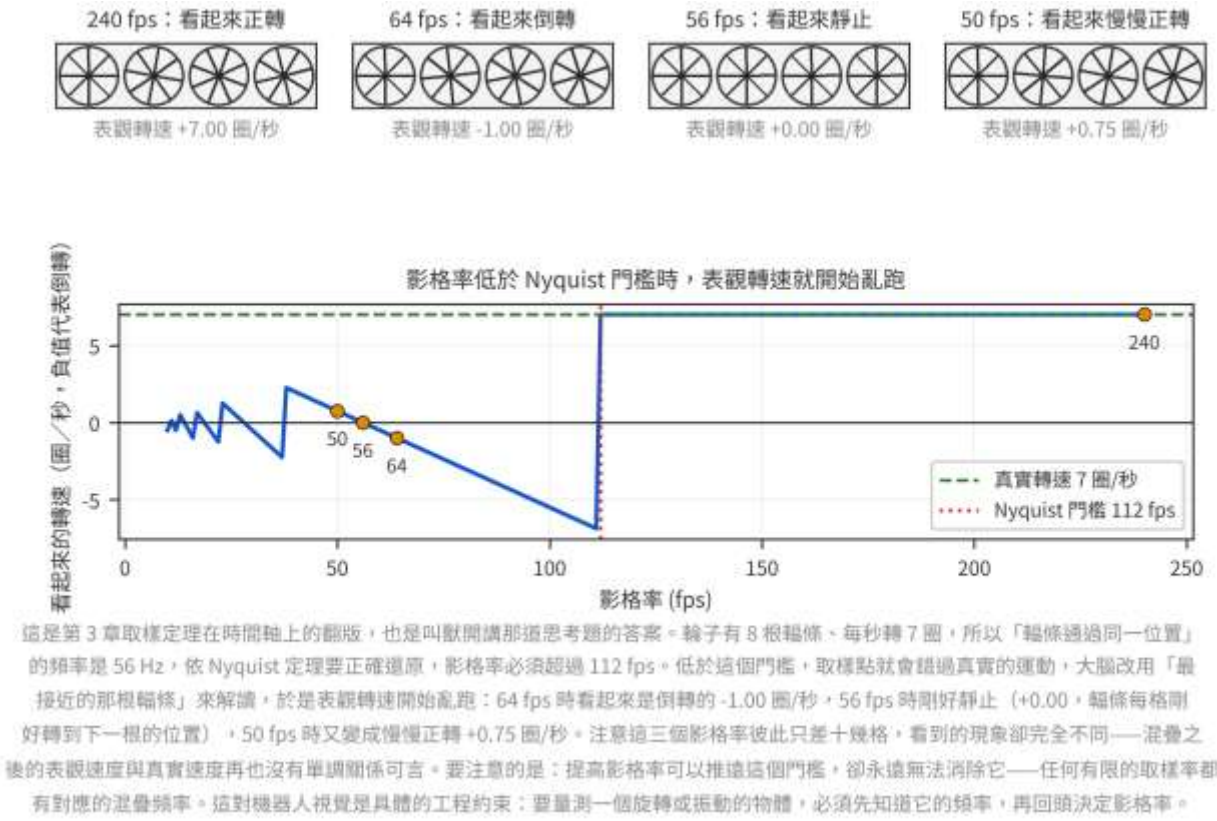


圖 12-3　時間混疊：影格率不足時，車輪會倒轉

12.2 視訊的讀取、處理與寫出

OpenCV 以 VideoCapture 讀取視訊(檔案或即時攝影機)，以 VideoWriter 寫出。核心觀念很簡單：視訊處理就是一個迴圈——逐影格讀取、把每一影格當成一張影像套用前面十一章的任何技術、再逐影格輸出或顯示。



圖 12-4　每一影格只有幾毫秒：各章演算法的實測耗時

程式 12-1 視訊讀取、處理與寫出的基本框架

```
import cv2

# 開啟視訊來源:檔名 → 讀檔;數字 0 → 預設攝影機;網址 → 串流
cap = cv2.VideoCapture("traffic.mp4")   # 或 0,或 ESP32-CAM 串流網址

# 查詢視訊屬性
fps    = cap.get(cv2.CAP_PROP_FPS)
width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
print(f"影格率={fps}, 尺寸={width}x{height}")

# 準備寫出(fourcc 指定編解碼器)
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter("output.mp4", fourcc, fps, (width, height))

while True:
    ok, frame = cap.read()      # 逐影格讀取
    if not ok:                  # 讀完或讀取失敗
        break

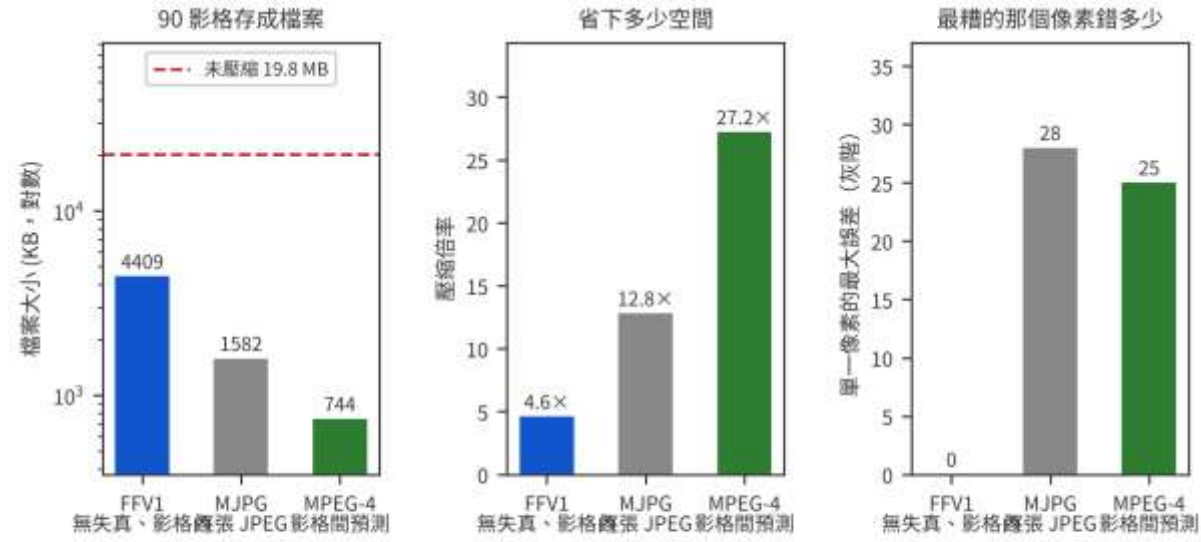
    # === 把每一影格當成一張影像:套用前 11 章的任何技術 ===
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 80, 160)          # 第 9 章
    result = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

    out.write(result)           # 寫出這一影格
    cv2.imshow("video", result)
    if cv2.waitKey(1) == 27:    # ESC 離開;waitKey 值影響播放速度
```

```
        break

cap.release()                       # 務必釋放資源
out.release()
cv2.destroyAllWindows()
```

幾個實務要點：第一，fourcc 是編解碼器的四字元代碼(如 mp4v、XVID、H264)，它決定輸出檔的壓縮方式，選錯會導致無法寫出或檔案過大——這直接關聯第 11 章的壓縮。第二，cap.read() 回傳的成功旗標一定要檢查，它是判斷「視訊是否讀完」的依據。第三，cv2.waitKey(n) 的 n 值(毫秒)控制每一影格之間的等待，影響顯示的播放速度;處理即時攝影機時通常設 1。第四，務必呼叫 release() 釋放資源，否則攝影機或檔案可能被鎖住。

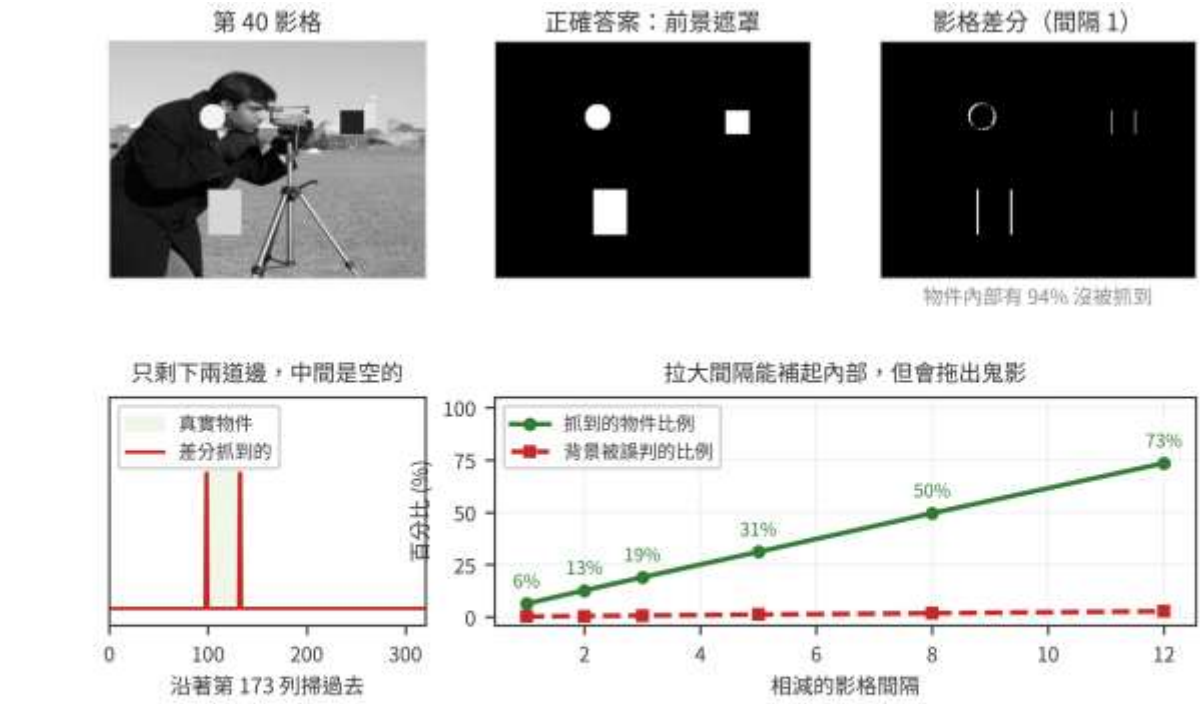


fourcc 那四個字母決定整段影片怎麼被壓縮，差別比多數人以為的大。同樣 90 張 320×240 的影格，未壓縮要 19.8 MB。三種策略各自代表一種取捨：FFV1 是無失真編碼，4409 KB、仍壓縮了 4.6 倍，而任何一個像素的誤差都是 0——存證與母帶就該用這一種;MJPG 把每一影格各自存成 JPEG，1582 KB、平均 36.8 dB，它完全不利用時間冗餘，但換來每一影格都能獨立解碼;mp4v 做影格間預測，只要 744 KB，不到 MJPG 的一半，品質 35.3 dB 也只差一點點。選擇的依據不是「哪個比較好」，而是你要什麼：要逐格剪輯與隨機存取就用 MJPG，要省空間就用影格間預測，要能當證據就用無失真。順帶一提，最大誤差這一欄比平均 PSNR 更能看出失真的性格——這正是第 11 章「失真的型態比失真的量更重要」那一課的延伸。

圖 12-5　編解碼器的三種策略：無失真、逐張、影格間預測

12.3 移動偵測：影格差分與背景相減

移動偵測是視訊處理最基礎也最實用的任務，它完美體現了「時間維度是資訊金礦」的觀念。核心思想只有一句：不變的是背景，變化的是前景。

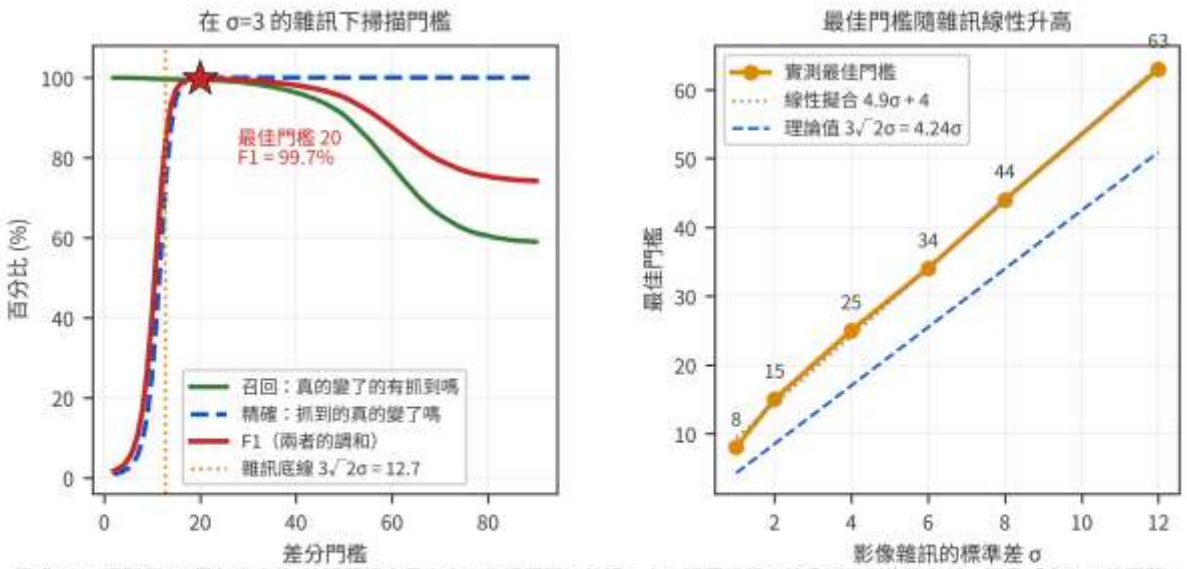


影格差分會得到「空心」的物件，這是初學者最常遇到的困惑，原因其實很單純：相減只會在「這一格與上一格不同」的地方留下數值。一個顏色均勻的物件，往右移動兩個像素之後，它的內部仍然蓋在同樣的灰階上，相減就是零——只有前緣（露出新背景）與後緣（蓋住舊背景）會留下痕跡。本圖實測：間隔 1 影格時，真實物件的面積有 94% 完全沒被抓到。把間隔拉大到 12 影格，抓到的比例從 6% 升到 73%，但背景的誤判率也從 0.2% 升到 2.7%——那些是物件「曾經在、現在不在」的位置，也就是鬼影。所以影格差分適合回答「有沒有東西在動」，卻不適合回答「那個東西長什麼樣、有多大」。要得到完整的物件，需要的是下一節的背景相減。

圖 12-6　影格差分為什麼是空心的

12.3.1 影格差分

最簡單的方法是影格差分(Frame Differencing)：把相鄰兩影格相減，取絕對值。靜止的背景在兩影格中相同，相減後為零(黑);移動的物件在兩影格中位置不同，相減後留下非零的差異(亮)。再對差值做門檻化(第 9 章)，就得到移動區域的遮罩。它的優點是極簡、極快、能自動適應緩慢的光線變化;缺點是：只能偵測到「正在移動」的部分，物件若暫停就消失;且移動物件的內部(顏色均勻處)前後影格相同，會被誤判為靜止，只留下物件的「邊緣輪廓」——這稱為「鬼影」或空心問題。



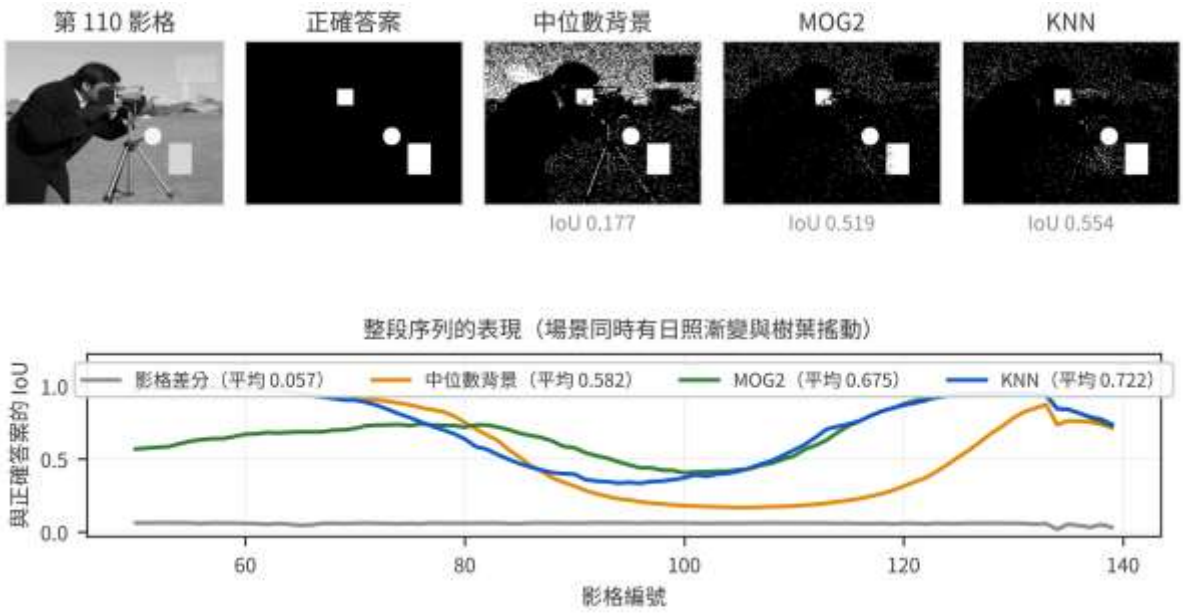
程式 12-2 裡那個 25 是怎麼來的?這張圖把它量出來。先把問題切乾淨：上一張圖說明了差分抓不到物件內部，那是「空心」的問題，不是門檻的問題。所以本圖的正確答案改用「乾淨序列中真的改變了的像素」（同一組合成參數、不加雜訊再跑一次），這樣量到的才是門檻真正在做的事：把真實變化與雜訊分開。結果：在 σ=3 的雜訊下最佳門檻是 20，F1 達到 99.7%（召回 99%、精確 100%）。右圖把雜訊從 σ=1 掃到 σ=12，最佳門檻從 8 升到 63，擬合出來是 4.9σ，與理論的 $3\sqrt{2}\sigma \approx 4.24\sigma$ 同一個量級（兩張含雜訊影格相減後，雜訊的標準差是 $\sqrt{2}\sigma$，取三倍即為可靠的底線）。這給了一條可操作的規則：門檻不該寫死，而該取一塊靜止區域量出 σ，再乘上約四倍。這也解釋了為什麼同一組參數換一台相機、換一個光照就失效。

圖 12-7　差分門檻怎麼挑：最佳門檻約為雜訊的四倍

12.3.2 背景相減

更強大的方法是背景相減(Background Subtraction)：先建立一個「純背景」的模型(沒有任何移動物件時的畫面)，再把每一影格與這個背景模型相減，差異顯著處就是前景。相較影格差分，它能偵測到完整的物件(而非只有邊緣)，且暫停的物件也不會消失。

關鍵難點在於「背景模型」如何建立與維護——真實場景的背景並非一成不變：光線會漸變(日照移動)、樹葉會搖動、水面會波動、攝影機會輕微震動。因此優秀的背景相減器必須是自適應的：它持續、緩慢地更新背景模型，把「長期穩定存在」的東西學習為背景，把「突然出現、快速移動」的東西判定為前景。OpenCV 提供的 MOG2(高斯混合模型)與 KNN 背景相減器就是這類自適應方法，它們為每個像素維護一個統計模型，能應付上述干擾，還能偵測並標記陰影。



這張圖把四種做法放在同一段序列上較量，而且場景刻意加了兩種真實世界的干擾：日照漸變（整體亮度緩慢起伏）與樹葉搖動（右上角一小塊持續抖動）。評分用 IoU——偵測到的前景與正確答案的交集除以聯集，1.0 代表完全吻合。影格差分平均只有 0.057，因為它永遠是空心的（上一張圖已經解釋過）。中位數背景 0.582：它能抓到完整物件，但背景模型一旦建好就不再更新，日照一變就整片誤判。MOG2 0.675 與 KNN 0.722 則持續緩慢更新模型，把「長期穩定存在」的學成背景、「突然出現」的判為前景，因此能同時應付漸變與抖動。本例中以 KNN 最好。這就是「自適應」三個字的具體意義：不是演算法比較聰明，而是它承認背景本來就會變。但也請注意三條曲線在中段同時明顯下滑——那正是日照變化最快的時候。自適應不等於免疫：模型更新需要時間，當場景變得比學習速度還快，誤判就無可避免。

圖 12-8　四種移動偵測在同一段序列上的正面交鋒

程式 12-2 影格差分與背景相減移動偵測

```
import cv2, numpy as np

cap = cv2.VideoCapture("surveillance.mp4")

# --- 方法一:影格差分 ---
ok, prev = cap.read()
prev_gray = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)

# --- 方法二:自適應背景相減器(實務首選) ---
backsub = cv2.createBackgroundSubtractorMOG2(
            history=500, varThreshold=16, detectShadows=True)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

while True:
    ok, frame = cap.read()
    if not ok:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 影格差分
    diff = cv2.absdiff(prev_gray, gray)
    _, motion = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
    prev_gray = gray

    # 背景相減 → 前景遮罩
```

```
    fgmask = backsub.apply(frame)

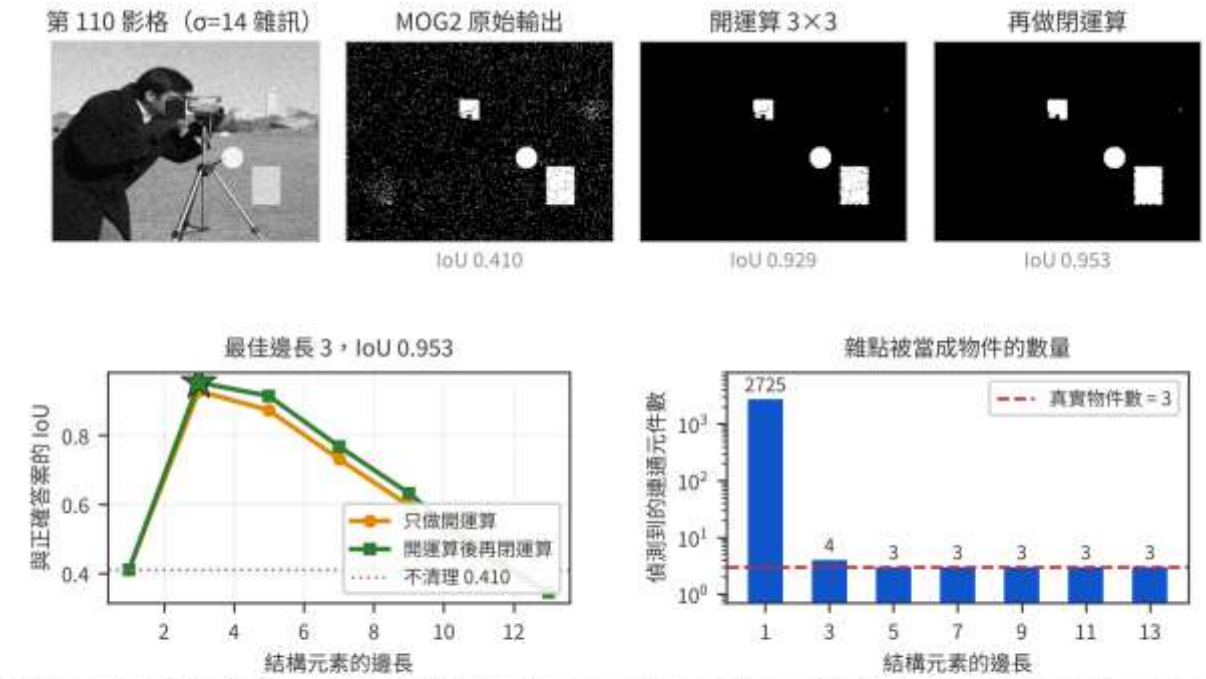
    # 型態學清理(第 10 章):去雜點、補破洞
    fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_OPEN, kernel)
    fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_CLOSE, kernel)

    # 連通元件框出移動物件(第 10 章)
    cnts, _ = cv2.findContours(fgmask, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
    for c in cnts:
        if cv2.contourArea(c) > 500:          # 面積門檻濾雜訊
            x, y, w, h = cv2.boundingRect(c)
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

    cv2.imshow("motion", frame)
    if cv2.waitKey(30) == 27:
        break

cap.release(); cv2.destroyAllWindows()
```

這段程式是本書多章的綜合：背景相減得到前景 → 第 10 章的型態學清理雜點與破洞 → 第 10 章的連通元件與第 9 章的輪廓框出物件。它是智慧監控、人流計數、交通流量分析、入侵偵測的骨幹，也正是第 9.7.4 節「視訊中的移動物件分割」的完整實作。



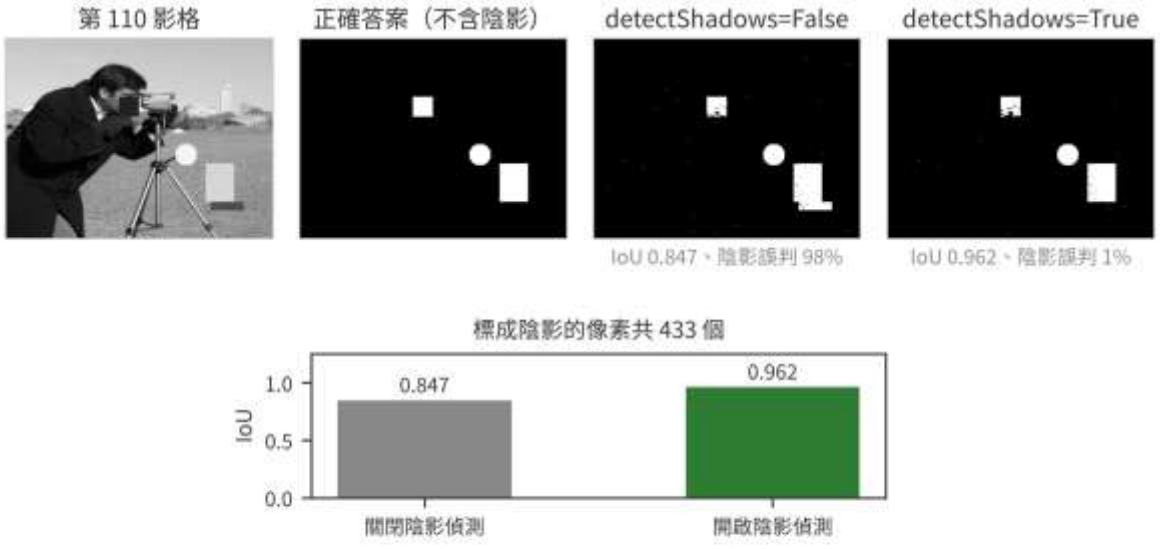
背景相減的輸出幾乎不可能直接用——它布滿雜點、物件內部有破洞。這正是第 10 章型態學派上用場的地方，而且順序有講究：先開運算（侵蝕再膨脹）去掉比結構元素小的雜點，再閉運算（膨脹再侵蝕）補起物件內部的破洞。本圖在 σ=14 的高雜訊下實測：未清理的 IoU 只有 0.410，開運算後升到 0.929，再閉運算達到 0.953。右圖是更具體的理由——未清理時連通元件多達 2725 個，而真實物件只有 3 個，也就是說若不清理就直接數物件，會得到荒謬的結果;邊長到 3 時才收斂到接近正確的數量。但也別把結構元素開得太大：邊長過大時 IoU 反而下降，因為小的物件連同雜點一起被侵蝕掉了。結構元素的大小應該由「最小的目標物件」決定，而不是由雜訊決定。

圖 12-9　型態學清理：接回第 10 章的開運算與閉運算

12.3.3 推導：背景模型的三個現實

背景相減在投影片上看起來很乾淨：建一個背景模型，相減，門檻化，完成。實際接上攝影機之後，會有三件事立刻找上門，而且都不是調參數就能繞過的。這一節把它們攤開來講，因為理解它們，比記住 MOG2 有哪些參數有用得多。

第一件是陰影。陰影跟著物件一起移動，所以背景相減理所當然地把它判為前景，框出來的物件因此比真實的大上一截——人流計數可能把一個人算成兩個，車速估計會算錯車尾的位置。陰影的判定靠的是一個很聰明的觀察：陰影只是讓像素「變暗」，但照到的還是同一塊地面，所以紋理與色度都沒變，只有亮度等比例下降。於是只要檢查「目前的像素值除以背景模型的像素值」是否落在一個固定區間內（例如 0.5 到 0.95）、且色度沒有明顯偏移，就能把它標成陰影。OpenCV 的 MOG2 把這件事做成 detectShadows 參數，開啟後輸出就變成三態：0 是背景、255 是前景、127 是陰影。這裡有一個很容易踩到的坑——程式裡必須寫 mask == 255，若沿用舊習慣寫 mask > 0，陰影會被當成前景，這個功能等於沒開。



陰影是移動偵測最常見的誤判來源，而且它有一個很討厭的性質：陰影會跟著物件一起移動，所以背景相減理所當然地把它判為前景，結果框出來的物件比真實大上一截（人流計數會把一個人算成兩個，車速估計會算錯車尾的位置）。MOG2 的 detectShadows 參數就是為此設計：它利用「陰影只是變暗、紋理不變」這個特性——若某個像素與背景模型的比值落在固定範圍內、且色度沒變，就標成陰影（輸出值 127 而非 255）。本圖實測：關閉時 IoU 0.847、陰影區有 98% 被誤判;開啟並濾掉 127 之後，IoU 升到 0.962、陰影誤判降到 1%，共有 433 個像素被正確標為陰影。要記得輸出值是三態的：0 是背景、255 是前景、127 是陰影，所以程式裡必須寫 mask == 255 而不是 mask > 0，否則這個功能等於沒開。

圖 12-10　陰影：跟著物件一起移動的那一塊誤判

第二件是停住的物件。很多教科書會說「背景相減的優點是暫停的物件不會消失」，這句話只對了一半。自適應的背景模型會持續把「長期穩定存在」的東西學成背景，所以一個物件只要停得夠久，它就會被吸收進背景，然後憑空消失。問題只在於「夠久」是多久——而這由學習率決定，也就是第三件事。

第三件是學習率，它是這三者的樞紐。MOG2 更新背景模型的方式大致是指數移動平均：$B \leftarrow (1-\alpha)\cdot B + \alpha\cdot I$，其中 α 就是學習率，I 是目前的影格。把這條式子展開，n 格之前的那一影格對現在的背景模型貢獻了 $\alpha(1-\alpha)^n$ 的權重，所以模型的「記憶長度」大約是 $1/\alpha$ 格——$\alpha = 0.01$ 記得住百來格，$\alpha = 0.05$ 就只記得住二十格左右。這一個數字同時決定了兩件相反的事：學得多快（暖身要幾格才把背景學乾淨），以及忘得多快（物件停住之後幾格會被吸收）。

圖 12-11 把這個取捨量了出來。學習率 0.05 時，背景模型只要 6 格就學乾淨了，但物件一停住，2 格之內就被吃掉;學習率 0.001 時，物件停住後可以撐 102 格不被吸收，代價是背景模型到第 170 格都還沒學乾淨。兩條曲線的形狀是對稱的，因為它們本來就是同一個 α 的兩面。

這個取捨決定了系統能不能用，而且答案完全取決於你要偵測什麼。要偵測「有沒有人經過」，學習率可以大一點，讓光線變化很快被吸收;但要偵測「有沒有人在門口徘徊」或「包裹被遺留在月台上」，學習率就必須非常小，甚至要另外維護一個更新極慢的長期背景模型，拿它與短期模型比對——短期模型認為是背景、長期模型認為是前景的區域，就是「最近才出現並停住」的東西。所以遺留物偵測在實務上是一個獨立的題目，而不是把背景相減的參數調一調就能解決。

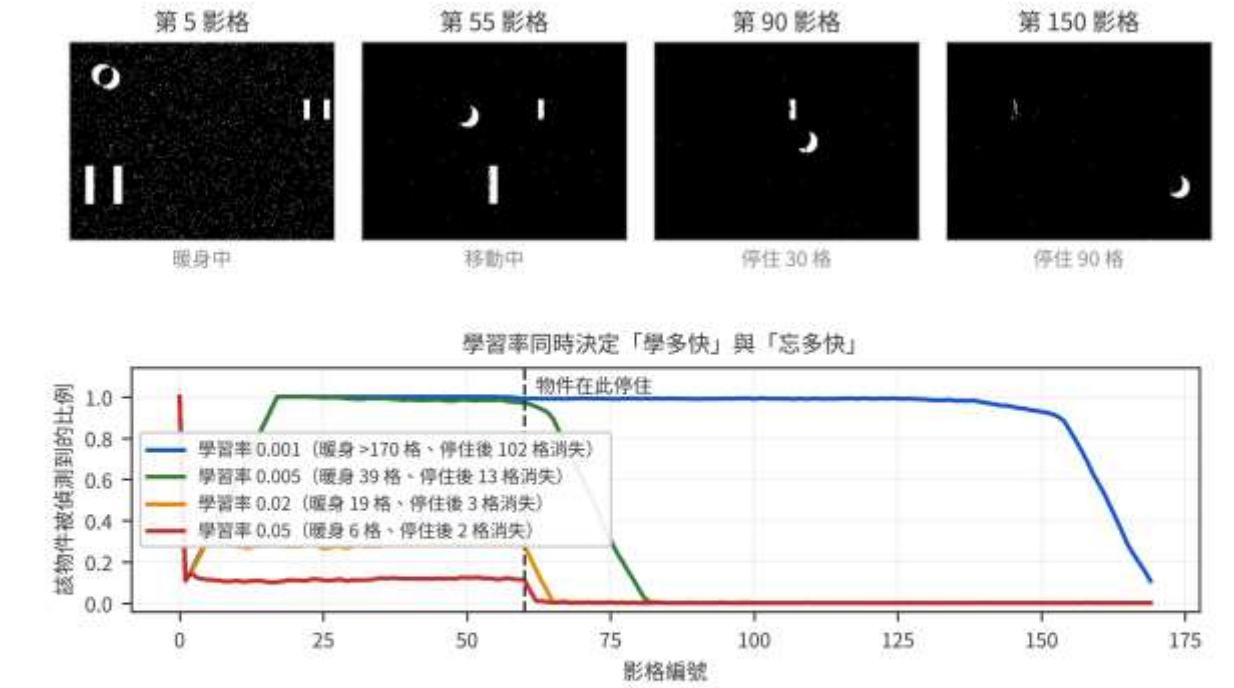
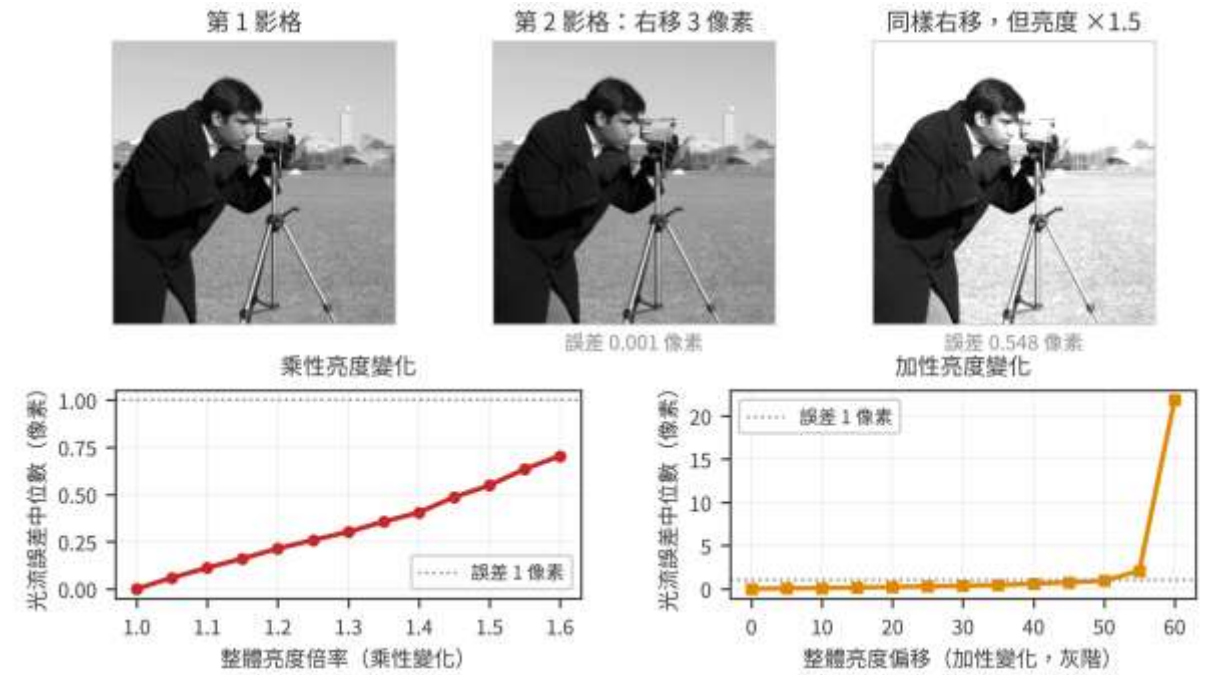


圖 12-11　學習率同時決定「學多快」與「忘多快」

12.4 光流：估計像素的運動

移動偵測只回答「哪裡有東西在動」，光流(Optical Flow)則更進一步，回答「每個點往哪個方向、以多快的速度移動」——它為影像中的點估計出運動向量。這是機器人理解動態世界的核心工具：知道了場景中每個點的運動，就能推算自身的移動、偵測逼近的障礙、預測物件的軌跡。

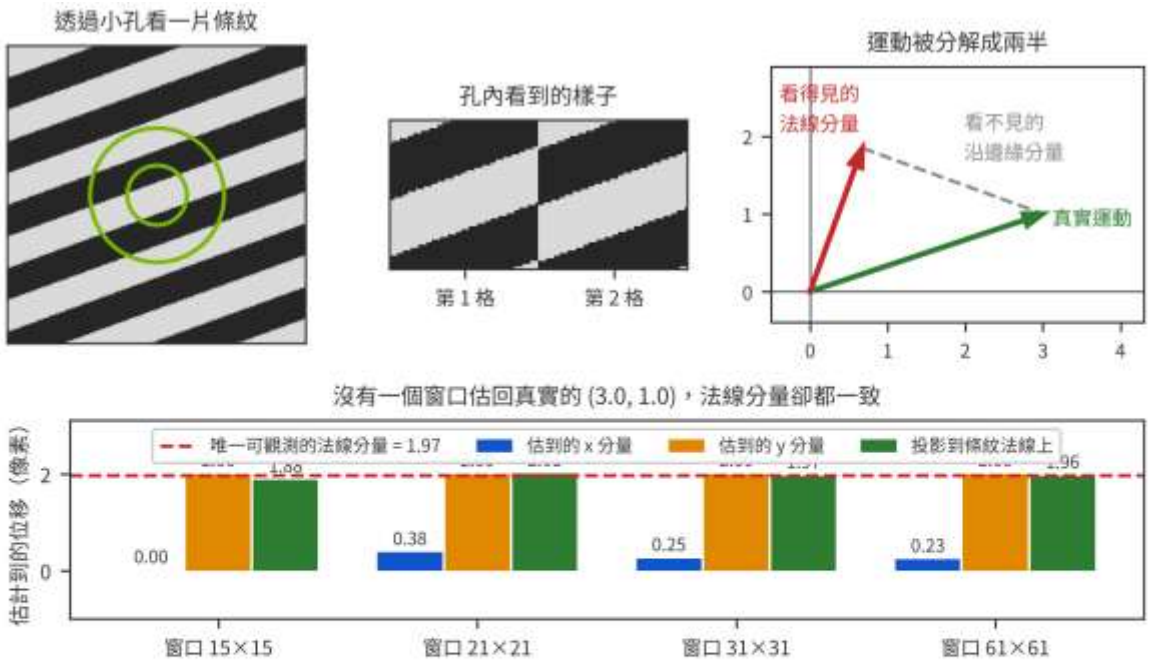


光流的全部推導都架在一個假設上：亮度恆定——同一個物體點在相鄰影格之間亮度不變，只是位置移動了。這個假設一旦被打破，演算法不會報錯，只會安靜地給出錯誤的答案，所以值得親自量一次它有多脆弱。本圖把同一張影像精確右移 3 個像素當作正確答案，再逐步破壞亮度：乘性變化（相當於光圈或快門改變）到 1.6 倍時，誤差從 0.001 像素升到 0.703;加性變化（相當於整體曝光補償）到 60 階時，誤差是 21.850 像素。兩者的差距大得出乎意料，而且方向與直覺相反：加性偏移的傷害是乘性的 31 倍。原因在光流方程式 Ix · u + Iy · v + It = 0 裡：整體加一個常數，空間梯度 Ix、Iy 完全不變，卻在每一點都塞進一個假的時間變化 It，於是解出來的速度被整個推歪;乘性變化則同時放大了空間梯度與時間變化，兩者在相除時大部分抵消掉了。這個對照值得記住：讓光流失效的不是「亮度變了多少」，而是「亮度的變化能不能被梯度解釋」。實務上的對策因此很具體：關掉相機的自動曝光與自動白平衡，或在算光流之前先做一次亮度正規化（第 5 章的直方圖等化或 CLAHE）。

圖 12-12　亮度恆定假設垮掉時，光流不會報錯，只會算錯

12.4.1 基本假設與孔徑問題

光流建立在一個核心假設上：亮度恆定——同一個物體點在相鄰兩影格之間，亮度不變，只是位置移動了。由這個假設可推導出光流方程式。但這裡有一個著名的困境：孔徑問題(Aperture Problem)。透過一個小孔觀察一條移動的直線邊緣，你只能看出它「垂直於邊緣方向」的運動分量，而沿著邊緣方向的運動則無從察覺(就像只透過吸管看一根滑動的長棍，無法判斷它沿長軸滑了多少)。因此，單看一個像素的局部資訊，無法唯一決定它的運動——必須引入額外的假設(例如鄰近像素運動相似)才能求解。

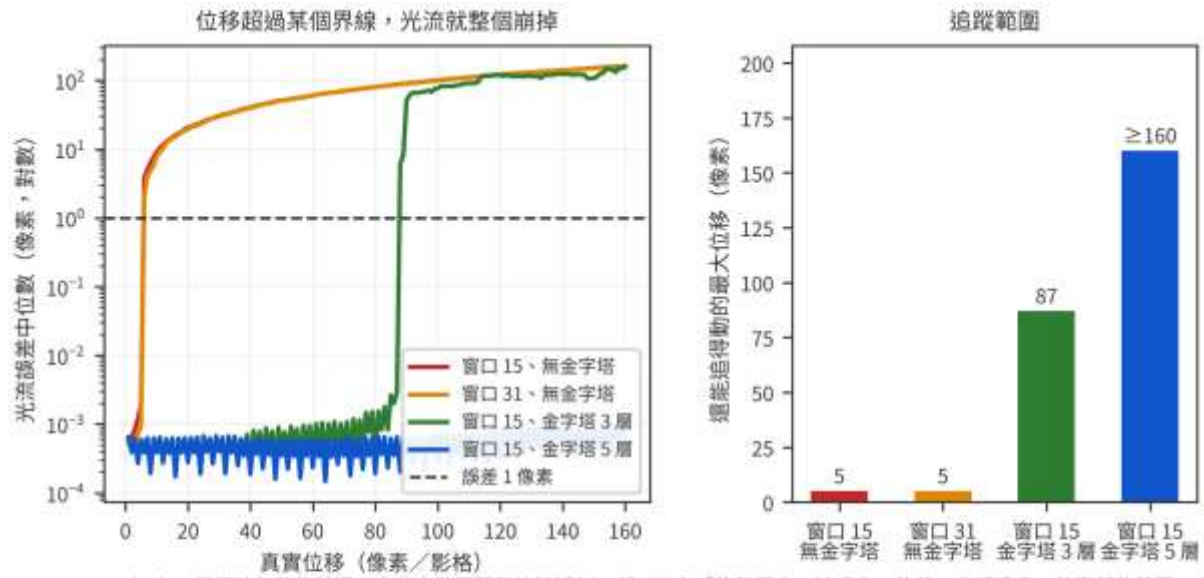


孔徑問題是光流最根本的限制：它不是演算法不夠好，而是資訊真的不在畫面裡。透過一個小孔看一片平行條紋移動，你只能看出條紋「垂直於自己」移動了多少;沿著條紋方向滑動多少，畫面一模一樣，因此無從得知。本圖的實測把這件事演得很清楚。同一對影格，只是改變 Lucas-Kanade 的窗口大小，估出來的位移就分別是 (0.00, 2.00)、(0.38, 2.00)、(0.25, 2.00)、(0.23, 2.00)——沒有一個等於真實的 (3.0, 1.0)，而且彼此也不一致。但把每一個都投影到條紋的法線方向，結果是 1.88、2.01、1.97、1.96，全部落在 1.97 附近。換句話說，演算法並沒有算錯：這些答案全都同樣正確，因為它們產生的畫面完全相同。真正被決定的只有法線分量，沿邊線的那一半是任意的，LK 只是從無限多組解中挑了一組交給你。這直接解釋了 12.4.2 節為什麼要挑角點：角點同時含有兩個方向的梯度，兩個未知數才有兩條獨立的方程式。邊緣上的點只能定出一個分量，平坦區則一個都定不出來——這和第 13 章判斷「哪些點適合當特徵」是同一套道理。

圖 12-13　孔徑問題：每個窗口的答案都不同，法線分量卻一致

12.4.2 稀疏光流：Lucas-Kanade

Lucas-Kanade 法計算稀疏光流：它不追蹤每一個像素，而是只追蹤少數「適合追蹤的特徵點」(通常用第 13 章的 Shi-Tomasi 角點——因為角點在兩個方向都有明顯變化，能繞過孔徑問題)。它假設一個小鄰域內的所有像素運動相同，藉此求解每個特徵點的運動向量。它高效、精準，適合追蹤特定的關鍵點，是視訊穩定、運動估計、視覺里程計(第 17 章機器人定位)的常用工具。

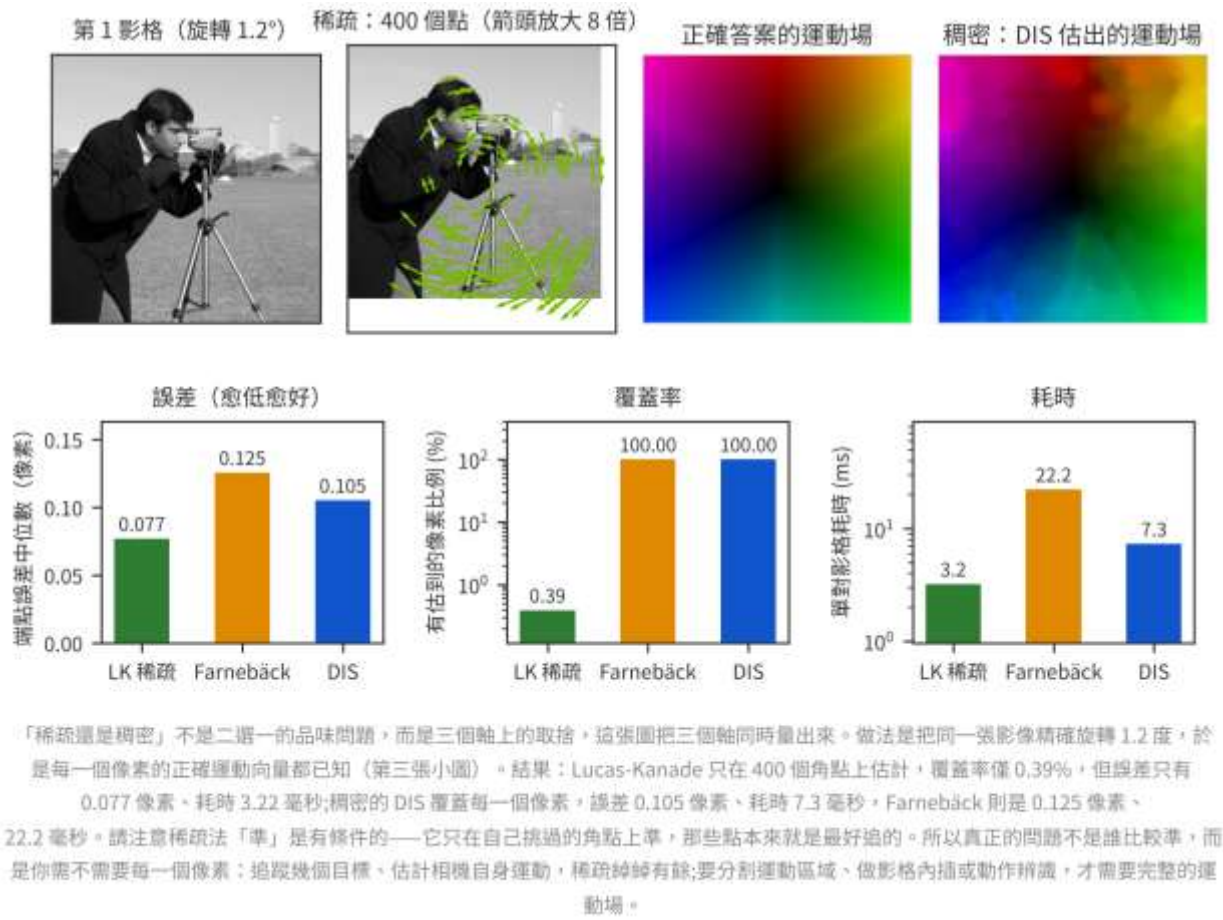


Lucas-Kanade 有一個常被忽略的前提：它用泰勒展開做線性近似，所以只在「位移很小」時成立。位移一旦超過窗口能涵蓋的範圍，它不是變得不準，而是完全失效。本圖把同一張影像精確平移已知的距離當正確答案，量出誤差中位數開始超過 1 像素的那個界線：窗口 15、無金字塔為 5 像素、窗口 31、無金字塔為 5 像素、窗口 15、金字塔 3 層為 87 像素、窗口 15、金字塔 5 層為 ≥160 像素（≥ 代表掃到本圖上限 160 像素仍未崩潰）。兩件事值得記住。第一，把窗口從 15 放大到 31 對追蹤範圍幾乎沒有幫助（5 對 5 像素）——因為限制來自線性近似的有效範圍，而那由影像本身的紋理尺度決定，不是把窗口開大就能改變的。第二，金字塔才是真正的解法——它先在縮小 8 倍的影像上估出粗略位移（此時 87 像素的位移看起來只剩幾個像素），再逐層放大修正，等於把「大位移」化約成一連串「小位移」。這又是一次多解析度分析（第 11 章）的勝利。實務上的推論很直接：若你的目標移動很快，與其調大窗口，不如增加金字塔層數，或者乾脆提高影格率。

圖 12-14　Lucas-Kanade 追得動多快：窗口沒用，金字塔才有用

12.4.3 稠密光流：Farnebäck

稠密光流(如 Farnebäck 法)則為影像中的每一個像素都計算運動向量，得到一張完整的「運動場」。它資訊更豐富(能看出整個場景的運動結構)，但運算量遠大於稀疏光流。常用第 8 章的虛擬色彩把運動場視覺化：用色相(H)表示運動方向、飽和度或明度表示運動速度，於是整個畫面的運動一目了然。稠密光流用於動作辨識、視訊分割、慢動作生成(影格內插)等需要全域運動資訊的任務。



「稀疏還是稠密」不是二選一的品味問題，而是三個軸上的取捨，這張圖把三個軸同時量出來。做法是把同一張影像精確旋轉 1.2 度，於是每一個像素的正確運動向量都已知（第三張小圖）。結果：Lucas-Kanade 只在 400 個角點上估計，覆蓋率僅 0.39%，但誤差只有 0.077 像素、耗時 3.22 毫秒;稠密的 DIS 覆蓋每一個像素，誤差 0.105 像素、耗時 7.3 毫秒，Farnebäck 則是 0.125 像素、22.2 毫秒。請注意稀疏法「準」是有條件的——它只在自己挑過的角點上準，那些點本來就是最好追的。所以真正的問題不是誰比較準，而是你需不需要每一個像素：追蹤幾個目標、估計相機自身運動，稀疏綽綽有餘;要分割運動區域、做影格內插或動作辨識，才需要完整的運動場。

圖 12-15　稀疏與稠密：誤差、覆蓋率、耗時三個軸上的取捨

程式 12-3 稀疏與稠密光流

```
import cv2, numpy as np

cap = cv2.VideoCapture("walking.mp4")
ok, prev = cap.read()
prev_gray = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)

# --- 稀疏光流:先找適合追蹤的角點(第 13 章 Shi-Tomasi) ---
pts = cv2.goodFeaturesToTrack(prev_gray, maxCorners=100,
        qualityLevel=0.3, minDistance=7)

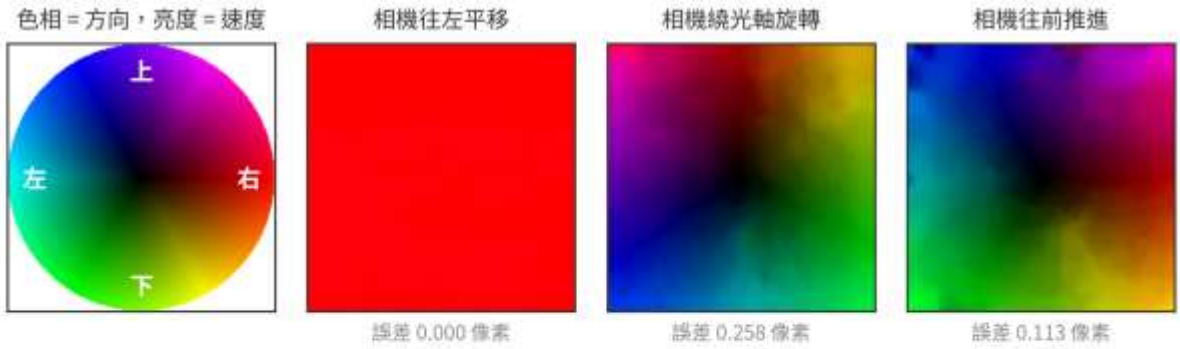
while True:
    ok, frame = cap.read()
    if not ok:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Lucas-Kanade 稀疏光流:追蹤這些點到新位置
    new_pts, status, err = cv2.calcOpticalFlowPyrLK(
        prev_gray, gray, pts, None)
    for (nx, ny), (ox, oy) in zip(new_pts[status==1],
                                  pts[status==1]):
        cv2.line(frame, (int(nx),int(ny)), (int(ox),int(oy)),
                 (0,255,0), 2)
```

```
        cv2.circle(frame, (int(nx),int(ny)), 3, (0,0,255), -1)

    # --- 稠密光流:每個像素的運動,用色彩視覺化 ---
    flow = cv2.calcOpticalFlowFarneback(prev_gray, gray, None,
            0.5, 3, 15, 3, 5, 1.2, 0)
    mag, ang = cv2.cartToPolar(flow[...,0], flow[...,1])
    hsv = np.zeros_like(frame)
    hsv[...,0] = ang * 180 / np.pi / 2      # 方向 → 色相(第 8 章)
    hsv[...,1] = 255
    hsv[...,2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
    flow_vis = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    cv2.imshow("sparse", frame)
    cv2.imshow("dense", flow_vis)
    prev_gray = gray
    pts = new_pts[status==1].reshape(-1,1,2)
```



稠密光流每個像素有兩個數字，沒辦法直接畫成灰階圖，所以慣例是借用第 8 章的 HSV：把運動的方向編成色相、大小編成亮度。最左邊是對照用的色輪——往右是青色、往下是藍紫、往左是紅橙、往上是綠。學會讀它之後，整個場景的運動結構一眼就能看出來：相機往左平移時，整張圖是單一顏色（所有像素同方向同速）；繞光軸旋轉時，顏色繞著中心轉一圈，而且中心最暗（轉軸上不動）、邊角最亮（半徑最大、線速度最快）；往前推進時，顏色從中心向外放射，像素都在往外跑，中心那個不動的點就是「擴張焦點」（Focus of Expansion），它指出相機正在往哪裡前進——這是視覺里程計與障礙閃避的關鍵線索，也是 12.7.4 節機器人案例的核心。所以運動場的圖樣本身就帶著幾何意義：均勻色塊代表平移，漩渦代表旋轉，放射狀代表前進或後退。

圖 12-16　運動場的色彩怎麼讀：平移、旋轉、前進的三種圖樣

12.4.4 推導：孔徑問題的數學形式，與金字塔為什麼有效

先把光流方程式寫出來。亮度恆定假設說的是：同一個物體點在相鄰影格間亮度不變，也就是 I(x, y, t) = I(x + u, y + v, t + 1)。對右邊做一階泰勒展開並消去 I(x, y, t)，得到 $I_x \cdot u + I_y \cdot v + I_t = 0$，其中 I_x、I_y 是空間梯度、I_t 是時間梯度。這條式子就是整個光流理論的起點，而它一眼就看得出問題：一條方程式，兩個未知數（u 與 v）。

這正是孔徑問題的數學形式。單一個像素提供的約束，只能把解限制在一條直線上——也就是只定出運動在梯度方向（邊緣的法線方向）的分量，沿著邊緣的那一個

分量完全沒有被約束。圖 12-13 用實測演了這件事：同一對影格，只改變 Lucas-Kanade 的窗口大小，估出來的位移分別是 (0.00, 2.00)、(0.38, 2.00)、(0.25, 2.00)、(0.23, 2.00)，沒有一個等於真實的 (3.0, 1.0);但把每一個都投影到條紋的法線方向，答案都是 1.96 左右。演算法沒有算錯——這些解產生的畫面完全相同，是資訊本身不足以分辨。

Lucas-Kanade 的補救是加上一個假設：一個小窗口內的所有像素具有相同的運動。於是窗口內的 N 個像素各給一條方程式，兩個未知數就有 N 條方程式，可以用最小平方法解。寫成矩陣形式，要解的是一個 2×2 的方程組，而那個 2×2 矩陣正是第 13 章 Harris 角點偵測用的同一個結構張量（由 I_x^2、I_xI_y、I_y^2 在窗口內的和組成）。這個矩陣可逆的條件，就是「兩個特徵值都夠大」，也就是角點的定義。所以 12.4.2 節說「要挑角點來追蹤」並不是經驗法則，而是這個方程組有唯一解的必要條件——平坦區兩個特徵值都接近零（矩陣奇異，一個分量都解不出來），邊緣上一個大一個接近零（只解得出法線分量）。

還有第二個限制，來自那次泰勒展開：一階近似只在位移很小的時候成立。位移一旦超過影像紋理的尺度，線性近似就完全失效——注意是失效，不是變得不準。直覺上會想「把窗口開大一點不就好了」，但圖 12-14 的實測說不行：窗口從 15 放大到 31，可追蹤的最大位移都是 5 個像素，一點也沒有改善。原因是限制來自近似的有效範圍，而那由影像本身的紋理尺度決定，把窗口開大只是納入更多同樣失效的方程式。

真正的解法是金字塔，而它的想法正是第 11 章的多解析度分析。先把兩張影格都縮小 2^n 倍，在最粗的那一層上，原本 80 個像素的位移看起來只剩幾個像素，線性近似重新成立，於是可以估出一個粗略的位移;把這個結果放大一倍當作下一層的起點，只需要再估一個小小的修正量;逐層下來，就等於把一個「大位移」化約成一連串「小位移」。圖 12-14 的實測：同樣的窗口 15，加上 3 層金字塔之後可追蹤的位移從 5 像

素跳到 87 像素，5 層則超過本圖掃描的 160 像素。實務上的推論很直接——若目標移動很快，與其調大窗口，不如增加金字塔層數（maxLevel），或者乾脆提高影格率把位移變小。

```
    if cv2.waitKey(30) == 27:
        break

cap.release(); cv2.destroyAllWindows()
```

12.5 物件追蹤

12.5.1 偵測與追蹤的差別

這是視訊分析中一組容易混淆卻至關重要的概念。偵測(Detection)是在單一影格中「找出物件在哪裡」，每一影格獨立進行，彼此無關;追蹤(Tracking)則是在連續影格中「維持同一個物件的身分」，把它在時間軸上的位置串成一條軌跡，並給它一個穩定的編號(ID)。

兩者為什麼要分工?因為各有優劣。偵測(尤其是第 16 章的 YOLO)準確但通常較耗算力，且它不知道「這一影格的人」和「上一影格的人」是不是同一個。追蹤則利用時間連續性，運算快(只需在上一影格位置附近搜尋)，還能維持身分——但它可能會「跟丟」(物件被遮擋、快速移動時)或「跟錯」(兩個物件交錯時身分互換)。

實務上最強健的方案是兩者結合，稱為「偵測式追蹤」(Tracking-by-Detection)：每隔幾影格用偵測器(YOLO)重新定位所有物件(校正累積誤差、發現新物件)，中間的影格則用輕量的追蹤演算法快速更新位置並維持 ID。著名的 SORT、DeepSORT 演算法就是這個框架，廣泛用於多物件追蹤(MOT)——這也是第 16 章 YOLO 追蹤功能的底層邏輯。

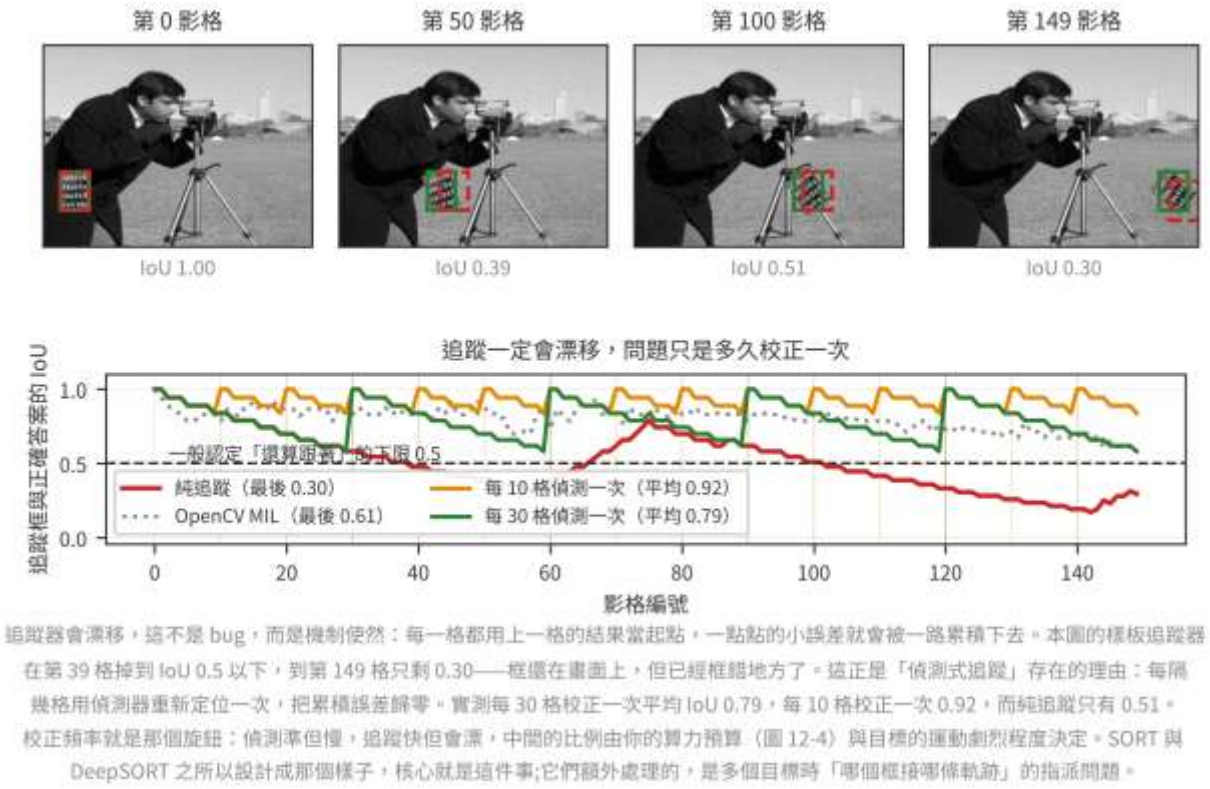


圖 12-17　追蹤一定會漂移，問題只是多久校正一次

12.5.2 追蹤演算法

傳統追蹤器種類繁多：MeanShift 與 CamShift 基於顏色直方圖(第 8 章)，追蹤顏色特徵明顯的物件;KCF、CSRT 等相關濾波器類追蹤器在速度與準確度間各有取捨(CSRT 較準、KCF 較快);卡爾曼濾波器(Kalman Filter)則從運動模型預測物件下一刻的位置，能在短暫遮擋時「猜測」物件的去向，是追蹤系統的經典組件。OpenCV 的 tracking 模組提供了這些追蹤器的現成實作。這裡要補一則現況提醒：KCF、CSRT、MOSSE 這些追蹤器一直住在 opencv_contrib 的 tracking 模組裡，而 OpenCV 5 的主套件(pip install opencv-python)並不包含它們，直接呼叫 cv2.TrackerCSRT_create() 會得到 AttributeError。主套件目前提供的是 TrackerMIL(傳統法)與 TrackerNano、TrackerVit、TrackerDaSiamRPN(以深度學習為基礎，需另外下載模型檔)。要用 KCF/CSRT 請改裝 opencv-contrib-python。在自己的環境上確認最快的方式是執行一行：print([x for x in dir(cv2) if x.startswith("Tracker")])。這類模組搬家在 OpenCV 的版本之間很常見，養成先查再

寫的習慣比記住 API 重要。

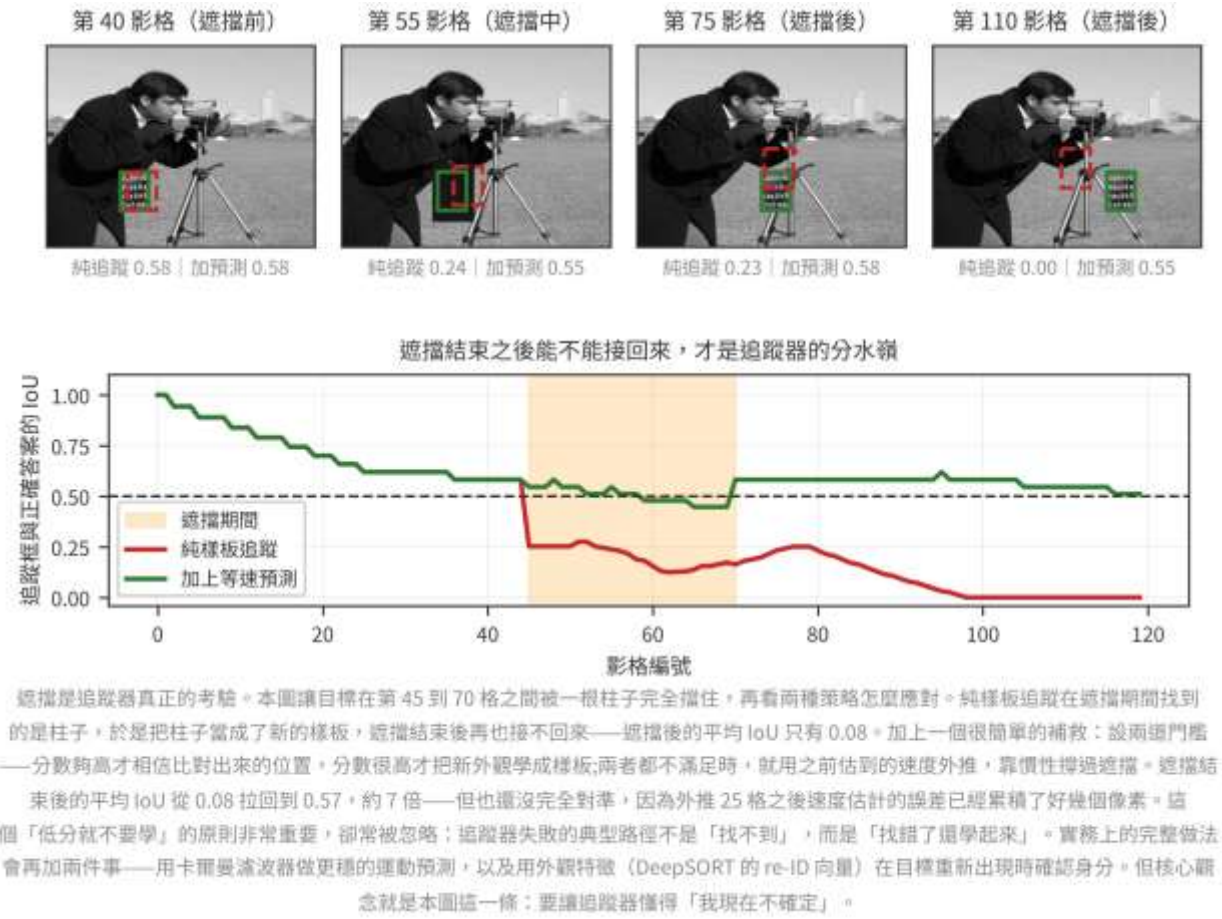


圖 12-18　遮擋：能不能接回來，才是追蹤器的分水嶺

程式 12-4 單物件追蹤

```
import cv2

cap = cv2.VideoCapture("target.mp4")
ok, frame = cap.read()

# 第一影格:框選要追蹤的物件(可手動或用第 16 章 YOLO 偵測)
bbox = cv2.selectROI("select", frame, False)

# 建立追蹤器(CSRT 較準,KCF 較快)
tracker = cv2.TrackerCSRT_create()
tracker.init(frame, bbox)

while True:
    ok, frame = cap.read()
    if not ok:
        break

    success, bbox = tracker.update(frame)   # 在新影格中更新位置
    if success:
        x, y, w, h = [int(v) for v in bbox]
```

```
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
    else:
        cv2.putText(frame, "Lost!", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

    cv2.imshow("tracking", frame)
```

12.5.3 推導：偵測多久該校正一次

追蹤器會漂移，這不是 bug，而是機制使然。追蹤的每一格都以上一格的結果為起點，所以每一格引入的小誤差不會被修正，只會被帶進下一格繼續累積。若每一格的位置誤差是獨立同分布、標準差為 σ 的隨機量，n 格之後累積誤差的標準差大約是 σ√n（隨機漫步）;而若誤差有系統性的偏向（例如樣板每格都往移動方向多學了一點背景），那就是 n·σ 的線性累積，更快。圖 12-17 的實測屬於後者：IoU 近乎線性地下滑，第 39 格就跌破 0.5，到第 149 格只剩 0.30——框還在畫面上，但已經框錯地方了。

偵測式追蹤的作用，就是每隔 k 格把累積誤差歸零一次。由此可以寫出一個粗略但好用的估算：若追蹤的漂移速率是每格 d 個像素，而你能容忍的最大偏移是 D 個像素，那麼校正週期 k 就該滿足 $k \le D / d$。圖 12-17 實測每 30 格校正一次的平均 IoU 是 0.79，每 10 格校正一次是 0.92，而純追蹤只有 0.51。另一邊是成本：偵測器（例如第 16 章的 YOLO）每次都要完整跑一遍，所以平均每格的運算量大約是「偵測成本 ÷ k + 追蹤成本」。k 就是那個旋鈕，把準確度與算力接在一起——這也是為什麼圖 12-4 的耗時實測值得放在本章的前面。

遮擋是另一個獨立的失敗模式，而且它的致命之處不在「找不到」，而在「找錯了還學起來」。當目標被擋住，比對分數最高的位置往往落在遮擋物上;若追蹤器照例把那裡的影像更新成新樣板，它就改追遮擋物了，遮擋結束後再也接不回來。圖 12-18 的實測：純樣板追蹤在遮擋結束後的平均 IoU 只剩 0.09。

補救的原則可以寫成一句話：要讓追蹤器懂得「我現在不確定」。具體做法是設兩道門檻——分數夠高才相信比對出來的位置，分數非常高才把新外觀學進樣板;兩者

都不滿足時，就不要更新任何東西，改用先前估到的速度把位置外推下去，靠慣性撐過遮擋。圖 12-18 實測這個簡單的補救讓遮擋後的平均 IoU 從 0.09 回到 0.57，約七倍;但也還沒完全對準，因為外推 25 格之後，速度估計的誤差已經累積了好幾個像素。

這正好指出實務系統還要補上的兩件事，也是 SORT 與 DeepSORT 的設計重點。其一是用卡爾曼濾波器取代土法外推：它用一個運動模型（通常是等速模型）持續預測下一刻的位置，並且會同時維護這個預測的不確定度，遮擋愈久、搜尋範圍就自動放得愈寬。其二是用外觀特徵確認身分：DeepSORT 為每個目標存一個再識別（re-ID）向量，目標重新出現時用它來確認「這是不是原來那一個」，而不是只看位置接不接得上。多目標時還有第三件事——「哪個偵測框該接哪一條軌跡」是一個指派問題，通常用匈牙利演算法求解。但這些都建立在本節這一條原則上：先承認自己不確定，才談得上接回來。

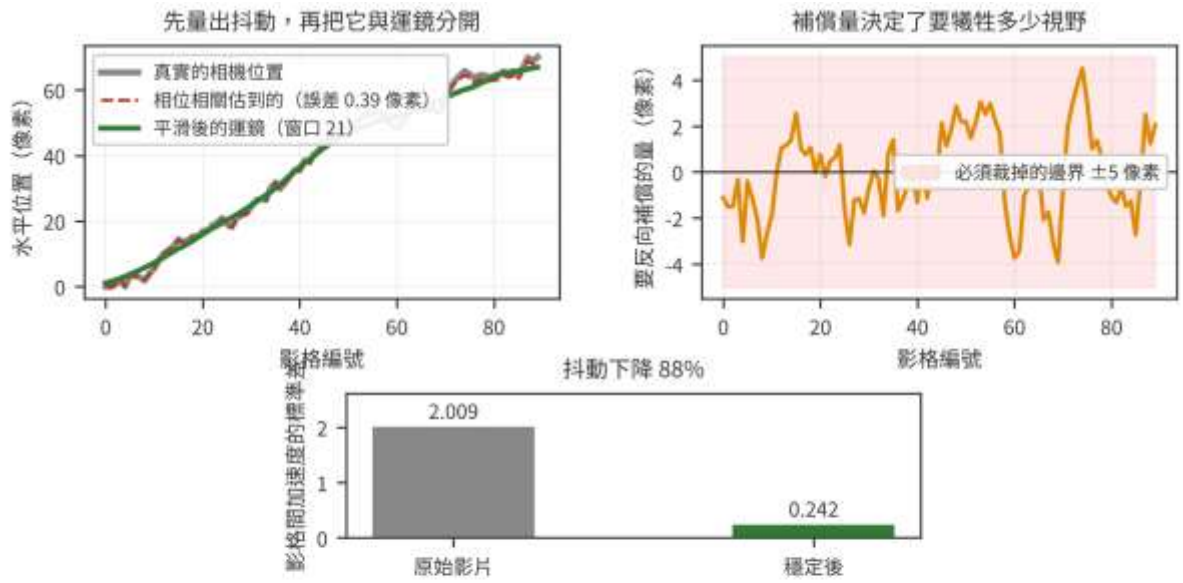
```
    if cv2.waitKey(30) == 27:
        break

cap.release(); cv2.destroyAllWindows()
```

12.6 進階主題：穩定、動作辨識與時序深度學習

12.6.1 視訊穩定

手持或車載攝影機的畫面總是晃動的。視訊穩定(Video Stabilization)的原理是：估計相鄰影格之間因抖動造成的整體運動(用第 6.7.2 節的相位相關法，或第 12.4 節的光流，或第 13 章的特徵匹配)，再反向補償這個運動(用第 4 章的幾何轉換把每一影格「拉回」穩定位置)，最後裁掉補償後露出的黑邊。這串起了第 4、6、12、13 章——是本書多項技術的綜合應用。



視訊穩定是本章與前面好幾章的綜合，流程只有三步，但每一步都有講究。第一步「量」：用第 6 章的相位相關估出相鄰影格的整體位移，本例的估計誤差中位數只有 0.39 像素。第二步「分」——這一步最關鍵：不是把所有運動都消掉，而是要把「刻意的運鏡」與「不要的手震」分開。做法是對軌跡做時間上的低通濾波（本圖用窗口 21 的移動平均），平滑後的那條就是攝影師想要的運鏡，兩者的差就是該補償掉的抖動。若不做這一步、直接把位移全部歸零，畫面會變成完全靜止，運鏡也沒了。第三步「補」：對每一影格做反向的幾何轉換（第 4 章），實測影格間加速度的標準差從 2.009 降到 0.242，下降 88%。代價寫在右圖：補償會把畫面推出邊界，所以必須裁掉 ±5 像素的邊框。這就是手機「超級防手震」會讓視野變窄的原因——穩定度與視野是同一條線上的兩端。

圖 12-19　視訊穩定的三步：量、分、補

12.6.2 動作辨識

動作辨識(Action Recognition)要回答「影片中的人在做什麼」(走路、揮手、跌倒)。它與單張影像分類的根本差別在於：動作是時間性的——單看一張「手舉起來」的照片，無法分辨是在打招呼還是要打人，必須看時間序列。傳統方法結合外觀特徵與光流(運動資訊);深度學習方法則用能處理時間維度的網路(如 3D 卷積、雙流網路、或後來的時序 Transformer)。應用包括：照護場景的跌倒偵測、運動分析、人機協作的手勢指令、異常行為監控。

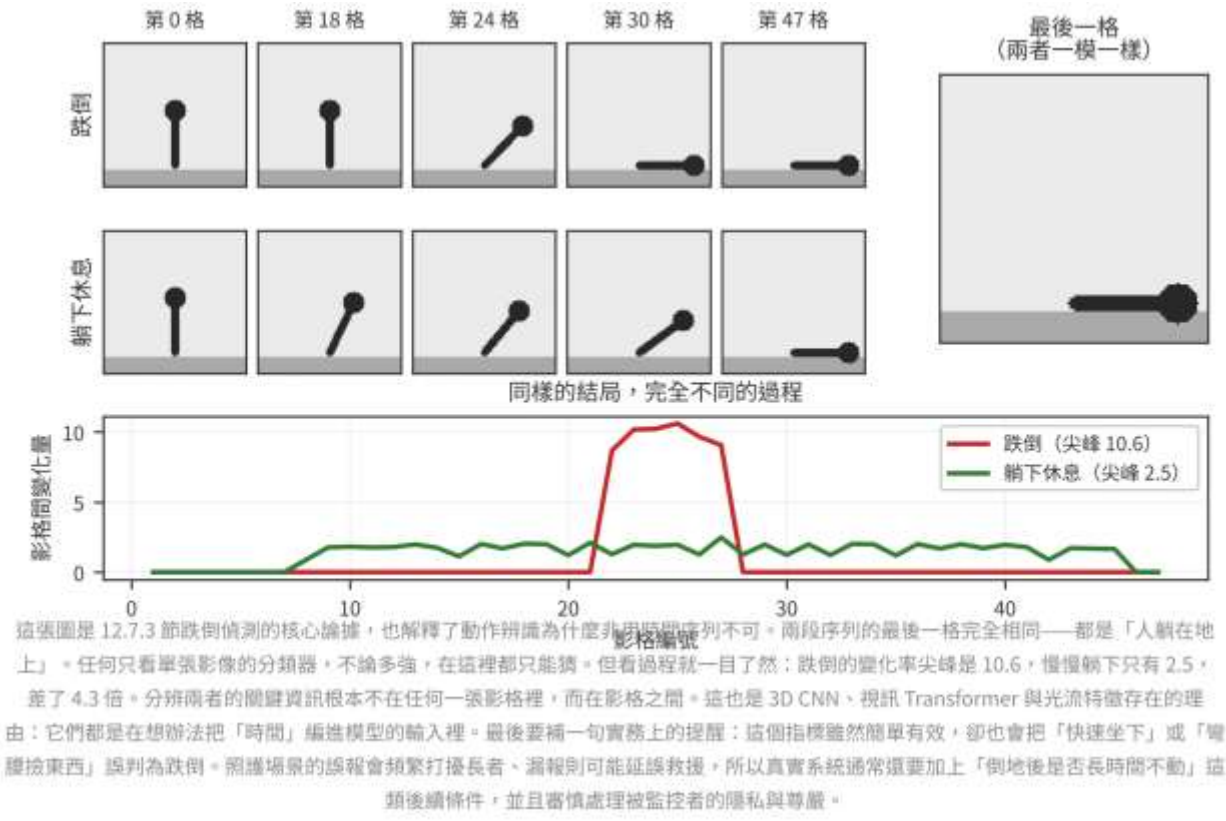


圖 12-20　同樣的結局，完全不同的過程：動作必須看時間序列

12.6.3 時序深度學習與時間一致性

進入深度學習，處理時間維度成為一個核心課題。早期用循環神經網路(RNN/LSTM)處理影格序列;3D CNN 把卷積從二維空間擴展到「空間加時間」的三維;近年的視訊 Transformer 則以注意力機制捕捉長距離的時間關聯。這些方法讓機器不只看懂單一影格，更能理解「事件的發展」。

但無論用什麼方法，視訊深度學習都要面對 12.1.3 節提過的時間一致性挑戰：若對每一影格獨立套用模型，結果會閃爍抖動(這一影格偵測到、下一影格又消失)。解決之道包括：在時間軸上做平滑、用追蹤維持身分、或設計本身就考慮時間關聯的模型。這也是機器人視覺與靜態影像辨識最大的不同——機器人活在連續的時間裡，它需要的不是一張張孤立的判斷，而是對「正在發生什麼」的連貫理解。

12.7 應用案例

12.7.1 案例一：智慧監控與人流計數

情境：在店門口或走廊上方架設攝影機，自動統計進出的人數。流程：背景相減得到移動前景(12.3 節)→ 型態學清理、連通元件框出每個人(第 10 章)→ 用追蹤器維持每個人的 ID(12.5 節)→ 在畫面中畫一條虛擬計數線，當某人的軌跡越過此線，依方向計數(進或出)。關鍵在於追蹤——若只靠偵測，同一個人可能被重複計數;有了穩定的 ID，才能確保「一個人只算一次」。這是零售分析、場館人流管理、防疫人數管制的實用系統。

12.7.2 案例二：車流量與車速偵測

情境：道路監控攝影機自動統計車流量與估計車速。流程：背景相減或第 16 章 YOLO 偵測車輛 → 追蹤每輛車的軌跡 → 由軌跡在連續影格的位移，結合第 4 章相機校正得到的「像素對應實際距離」的比例，以及已知的影格率(位移除以時間即速度)，換算出每輛車的實際速度。這串起了第 4 章(相機校正把像素換算成公尺)、第 12 章(追蹤取得位移)、以及影格率(提供時間基準)——是一個把「像素運動」轉換成「物理速度」的漂亮範例，也是智慧交通、區間測速的基礎。

12.7.3 案例三：照護場景的跌倒偵測

情境：在長者居家或病房中，自動偵測跌倒並發出警示。這是動作辨識(12.6.2 節)的重要應用，也凸顯了「時間維度不可或缺」——單看一張「人躺在地上」的照片，無法分辨是跌倒還是正常躺下休息;必須看時間序列：一個「快速的、由站立到倒地的垂直運動」才是跌倒的特徵。傳統做法可結合人體姿態(第 16 章 YOLO Pose)的重心高度變化與運動速度來判斷;深度學習則用時序模型直接學習跌倒的動作模式。此案例同時觸及第 16 章的姿態估計，展示了視訊、姿態、時序三者的整合。

這裡也值得提醒一個貫穿全書的價值：照護應用直接關係人的安危與尊嚴，系統的誤報(把正常動作誤判為跌倒，頻繁打擾)與漏報(真跌倒卻沒偵測到)都有嚴重後果，且涉及被監控者的隱私。這類系統的設計，技術之外更需要對使用者處境的體貼——

這是工程倫理的一環。

12.7.4 案例四：機器人的視覺里程計

情境：一台自主移動機器人(或無人機)需要知道自己移動了多少、轉了多少，即使沒有 GPS。視覺里程計(Visual Odometry)的核心正是本章的光流與追蹤：機器人前進時，攝影機看到的靜態場景會「反向流動」;透過追蹤場景中的特徵點(第 13 章)、分析它們在連續影格間的運動(12.4 節光流)，就能反推出攝影機自身的運動——這是機器人「自我定位」的視覺基礎，也是第 17 章機器人視覺整合的重要前置技術。這個案例揭示了一個深刻的觀點：同樣是「場景中的運動」，換個角度看，就成了「觀察者自身的運動」——運動是相對的，而機器人正是利用這個相對性來認識自己在世界中的位置。

12.8 本章與全書的觀念連結總表

相關章節	共用的觀念	視訊處理中的角色
第 3 章 數位影像基礎	取樣定理	影格率不足造成時間混疊(車輪倒轉錯覺)
第 4 章 幾何轉換	仿射/透視轉換、相機校正	視訊穩定的運動補償;像素換算實際速度
第 5 章 空間域強化	多影格平均	夜景模式即時間維度的去雜訊
第 6 章 頻率域	相位相關法	視訊穩定的影格間運動估計
第 8 章 色彩處理	HSV、顏色追蹤、虛擬色彩	稠密光流視覺化;CamShift 顏色追蹤
第 9、10 章 分割/二值	門檻、型態學、連通元件	前景遮罩的清理與物件框選
第 11 章 小波	壓縮	視訊編碼進一步利用影格間冗餘

第 13 章 特徵擷取	角點、特徵匹配	Shi-Tomasi 角點供稀疏光流追蹤
第 16 章 物件偵測	YOLO、姿態估計	偵測式追蹤;跌倒偵測結合姿態與時序
第 17 章 邊緣部署	即時性、視覺里程計	視訊處理的算力預算;機器人自我定位

表 12-1 第 12 章與全書其他章節的觀念連結

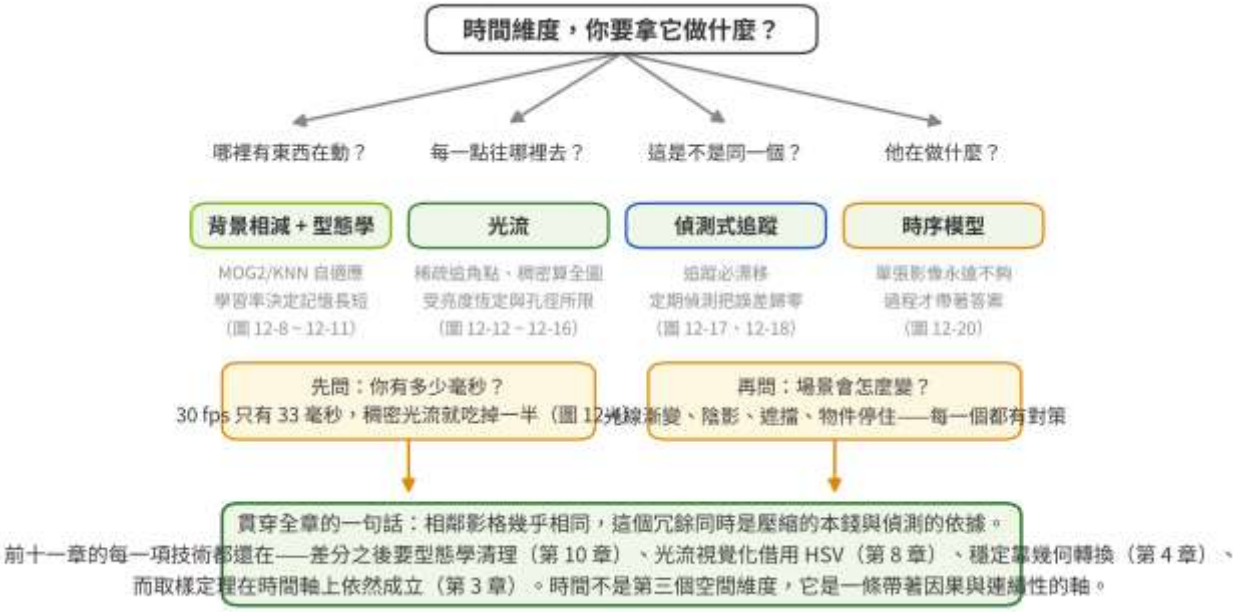


圖 12-21　第 12 章的決策地圖

程式實作 Lab 12

Lab 12-1 視訊處理管線

目標：熟悉視訊讀寫框架。(a)以程式 12-1 為基礎，讀取一段視訊或即時攝影機，對每一影格套用前面各章的至少三種技術(如 CLAHE、Canny、HSV 顏色分割)，並排顯示原始與處理結果;(b)在畫面上即時顯示影格率(用 cv2.getTickCount 計時)，觀察不同處理的耗時;(c)把結果寫成輸出檔，嘗試不同的 fourcc 編解碼器，比較輸出檔的大小與品質;(d)討論題：哪一種處理最耗時?若要在 30 fps 即時執行，每一影格的處理時間預算是多少毫秒?

Lab 12-2 移動偵測監控系統

目標：實作 12.7.1 節的監控系統。(a)以程式 12-2 對一段監控影片做背景相減，比較「影格差分」與「MOG2 背景相減」在偵測完整物件、應對暫停物件、抵抗光線變化上的差異;(b)加入型態學清理與面積門檻，穩定地框出移動物件;(c)畫一條虛擬計數線，實作越線計數(需簡單追蹤以避免重複計數);(d)測試干擾：讓畫面中有搖動的樹葉或漸變的光線，觀察 MOG2 如何自適應;(e)挑戰題：加入陰影偵測(detectShadows)，避免把物體的影子也框進去。

Lab 12-3 光流視覺化

目標：直觀理解光流。(a)以程式 12-3 對一段有明顯運動的影片(如行走的人、車流)，同時顯示稀疏光流(特徵點的運動軌跡)與稠密光流(色彩運動場);(b)解讀稠密光流的色彩圖：不同顏色代表什麼方向?為什麼靜止的背景是黑色?(c)拍攝或找一段「攝影機自己在移動」的影片(如手持前進)，觀察整個畫面的光流呈現什麼模式——這與 12.7.4 節的視覺里程計有何關聯?(d)驗證孔徑問題：對著一條緩慢移動的直線邊緣，觀察光流估計是否只捕捉到垂直於邊緣的分量。

Lab 12-4 物件追蹤與偵測式追蹤

目標：體會偵測與追蹤的分工。(a)以程式 12-4 用 CSRT 追蹤器追蹤一個手選的物件，測試它在物件被短暫遮擋、快速移動、與其他物件交錯時是否跟丟或跟錯;(b)比較 KCF 與 CSRT 在速度與準確度上的差異(量測 FPS);(c)概念實作偵測式追蹤：每隔 N 影格「重新框選」(模擬偵測器校正)，中間影格用追蹤器更新，觀察這樣是否比純追蹤更穩健;(d)討論題：承第 16 章，若把手動框選換成 YOLO 自動偵測，並對多個物件各自維持 ID，你會如何設計?(這正是 DeepSORT 的思路)

Lab 12-5 自己做一段有正確答案的測試序列

目標：學會「造一個有正確答案的資料集」——這是本章最重要的方法論訓練。真實影片沒有標準答案，調參數只能靠眼睛看，但只要序列是自己合成的，每一格的

物件位置與遮罩都已知，就能真的算出誤差。(a)寫一個函式，把一張靜態影像當背景，讓一個方塊、一個圓以已知的速度移動，同時回傳每一格的前景遮罩當正確答案;(b)把雜訊加在最後一步，並讓函式接受一個亂數種子，使得同一個種子配不同的雜訊強度會得到「幾何完全對齊」的乾淨版與含雜訊版（這一點在後面會很有用）;(c)加上三種真實世界的干擾選項：整體亮度緩慢起伏（日照漸變）、某一小塊持續隨機抖動（樹葉搖動）、物件底下跟著一塊變暗的區域（陰影）;(d)再加一個選項讓物件在指定影格停住不動;(e)用你的序列重做圖 12-7 的門檻掃描：用乾淨版算出「真的改變了的像素」當正確答案，掃描門檻並畫出召回率、精確率與 F1，找出最佳門檻;(f)把雜訊強度從 σ=1 掃到 σ=12，畫出最佳門檻對 σ 的關係，並與理論值 3√2σ 比較。你得到的係數是多少?

Lab 12-6 偵測式追蹤與遮擋

目標：親手量出追蹤的漂移，並體會「讓追蹤器承認自己不確定」的威力。(a)沿用 Lab 12-5 的序列產生器，但把目標換成有紋理的影像區塊（例如從另一張照片裁一小塊）。想一想：為什麼顏色均勻的方塊不適合用來測試追蹤器?（提示：回頭看 12.4.4 節的孔徑問題）;(b)用 cv2.matchTemplate 寫一個最小的樣板追蹤器：每一格在上一格位置附近搜尋，找到後更新樣板。讓目標同時做次像素位移與緩慢自轉，再畫出 IoU 對影格的曲線，記錄它在第幾格跌破 0.5;(c)加入「每 k 格用正確答案重新定位一次」模擬偵測式追蹤，對 k = 10、30 各畫一條曲線，比較平均 IoU。你應該會看到鋸齒狀的曲線——解釋每一個鋸齒的形狀;(d)估算：若你的偵測器要 40 毫秒、追蹤器要 2 毫秒，在 30 fps 的預算下 k 最小可以設到多少?(e)讓目標在中途被一根柱子完全擋住 25 格。先用原本的追蹤器跑一次，觀察遮擋結束後能不能接回來;(f)改成兩道門檻：分數夠高才相信位置、分數非常高才更新樣板，兩者都不滿足時用先前估到的速度外推。比較兩者在遮擋後的平均 IoU，並說明外推能撐多久、為什麼會慢慢失準;(g)選作：把等速外推換成卡爾曼濾波器（cv2.KalmanFilter），觀察它在遮擋期間如何同時給出預測位置與不確定度。

本章重點整理

- 視訊是以固定影格率排列的連續影格，多了「時間」維度;視訊處理的基本框架是逐影格讀取、當成影像處理、逐影格輸出的迴圈。時間維度帶來偵測移動、追蹤、估計速度、多影格提質的機會，也帶來即時性、時間一致性、資料量三大挑戰。
- 移動偵測基於「背景不變、前景變化」：影格差分極簡但只偵測到移動邊緣、暫停物件會消失;背景相減能偵測完整物件，MOG2/KNN 等自適應方法可應對光線漸變與搖動干擾，但自適應是雙面刃——物件只要停得夠久就會被學進背景而消失，撐多久由學習率決定（12.3.3 節、圖 12-11）。
- 光流估計每個點的運動向量，基於亮度恆定假設，受孔徑問題限制;Lucas-Kanade 為稀疏光流(追蹤角點),Farnebäck 為稠密光流(每像素運動場，常以虛擬色彩視覺化)。
- 偵測是單一影格找物件、追蹤是跨影格維持身分;偵測式追蹤(SORT/DeepSORT)結合兩者——定期偵測校正、中間追蹤維持 ID;傳統追蹤器有 MeanShift/CamShift(顏色)、KCF/CSRT(相關濾波)、卡爾曼濾波(運動預測)。
- 視訊穩定綜合光流/特徵匹配估計抖動、以幾何轉換反向補償;動作辨識需時間序列(單張影像無法分辨動作);時序深度學習(3D CNN、視訊 Transformer)須處理時間一致性以免結果閃爍。
- 視訊處理串起全書：第 3 章取樣(時間混疊)、第 4 章幾何(穩定與速度換算)、第 8 章色彩(光流視覺化)、第 9/10 章(前景清理)、第 13 章(角點追蹤)、第 16 章(偵測式追蹤、跌倒偵測)、第 17 章(視覺里程計)。

習題

1. 視訊相較於單張影像，多了什麼維度?這個維度帶來哪些機會與哪些挑戰?

提示與參考解答：多了「時間」維度。機會：相鄰影格高度關聯，可偵測移動(影格相減)、追蹤物件、估計速度、多影格提質去雜訊。挑戰：即時性(須跟上每秒數十影格)、時間一致性(結果不可影格間抖動閃爍)、資料量(數十倍於影像，仰賴壓縮與影格間編碼)。

2. 承叫獸開講的思考題：用影格率不足的相機拍攝高速旋轉的車輪，會看到車輪倒轉或靜止的錯覺，為什麼?這與第 3、6 章的什麼概念相關?

提示與參考解答：這是時間維度的混疊(時間 aliasing)。車輪旋轉是一個高頻的週期運動，影格率(取樣頻率)不足以滿足 Nyquist 定理時，取樣點恰好錯過真實運動，大腦把離散樣本腦補成較低頻的運動(倒轉或靜止)。這與第 3 章取樣定理、第 6 章混疊(高頻摺疊成低頻)同源，只是發生在時間軸上。

3. 視訊處理的基本框架是什麼?請說明 VideoCapture、逐影格迴圈、VideoWriter 三者的角色，以及 fourcc 與 release 的重要性。

提示與參考解答：框架是一個迴圈：VideoCapture 逐影格讀取來源(檔案/攝影機/串流)→ 每一影格當成影像套用任何影像處理技術 → VideoWriter 逐影格寫出。fourcc 指定輸出的編解碼器(決定壓縮方式，選錯無法寫出),release 釋放攝影機/檔案資源以免被鎖住。cap.read 的成功旗標用於判斷視訊是否讀完。

4. 影格差分與背景相減有何差異?各自的優缺點是什麼?為什麼影格差分會產生「空心」的移動物件?

提示與參考解答：影格差分：相鄰兩影格相減，極簡極快、自動適應光線，但只偵測到移動的邊緣，暫停物件會消失，且物件內部顏色均勻處前後影格相同、相減為零，故只留輪廓(空心/鬼影)。背景相減：與一個學習得到的背景模型相減，能偵測完整物件，且在物件短暫停住時仍看得見(這點勝過影格差分)，但需自適應維護背景模型以應對光線與搖動干擾(MOG2/KNN)。要注意「暫停物件不會消失」只對了一半：自適應模型會把長期靜止的東西學成背景，實測在學習率 0.05 時物件停住 2 格就被吸收、0.001 時可撐 102 格——撐多久完全由學習率決定(12.3.3 節、圖 12-11)。

5. 為什麼優秀的背景相減器必須是「自適應」的?它要應對哪些真實場景的干擾?

提示與參考解答：因為真實背景並非一成不變：日照移動使光線漸變、樹葉搖動、水面波動、攝影機輕微震動。自適應背景相減器持續緩慢更新背景模型，把「長期

穩定存在」的學為背景、「突然出現快速移動」的判為前景，並可偵測陰影，才能在這些干擾下正確運作。

6. 光流的核心假設是什麼?什麼是孔徑問題?它為什麼使單一像素的運動無法唯一決定?

提示與參考解答：核心假設是亮度恆定：同一物體點在相鄰影格間亮度不變、只是位置移動。孔徑問題：透過小孔(局部)觀察移動的直線邊緣，只能看出垂直於邊緣方向的運動分量，沿邊緣方向的運動無從察覺。故單看一個像素的局部資訊無法唯一決定運動，須引入額外假設(如鄰域運動相似)才能求解。

7. 稀疏光流與稠密光流有何差別?Lucas-Kanade 為什麼要選角點來追蹤?

提示與參考解答：稀疏光流(Lucas-Kanade)只追蹤少數適合追蹤的特徵點，高效精準;稠密光流(Farnebäck)為每個像素計算運動向量，資訊豐富但運算量大，常以虛擬色彩視覺化。Lucas-Kanade 選角點(Shi-Tomasi)是因為角點在兩個方向都有明顯灰階變化，能繞過孔徑問題、運動可唯一決定。

8. 「偵測」與「追蹤」有何根本差別?為什麼「偵測式追蹤」結合兩者會更強健?

提示與參考解答：偵測是在單一影格找出物件位置、影格間獨立無關;追蹤是跨影格維持同一物件的身分(ID)並串成軌跡。偵測準但耗算力、不知影格間是否同一物件;追蹤快且維持身分但可能跟丟或跟錯。偵測式追蹤(SORT/DeepSORT)定期用偵測器校正並發現新物件，中間影格用輕量追蹤維持 ID，兼得準確、效率與身分一致性。

9. 為什麼動作辨識(如跌倒偵測)必須用時間序列，而不能只看單張影像?

提示與參考解答：動作是時間性的：單看一張「人躺在地上」的照片無法分辨是跌倒還是正常躺下休息;單看「手舉起」無法分辨是打招呼還是攻擊。必須看時間序列——例如「快速的、由站立到倒地的垂直運動」才是跌倒特徵。故需結合光流/姿態的時間變化，或用 3D CNN、視訊 Transformer 等時序模型。

10. 什麼是視訊深度學習的「時間一致性」問題?若對每一影格獨立套用模型會發生什麼?如何緩解?

提示與參考解答：時間一致性指處理結果在影格與影格之間應穩定連貫。若每一影格獨立套用模型，結果會閃爍抖動(偵測框跳動、物件時有時無、ID 跳號)。緩解：在時間軸上平滑結果、用追蹤維持身分、或設計本身考慮時間關聯的模型(3D

CNN、時序 Transformer)。這是機器人視覺與靜態辨識最大的不同。

11. 視訊穩定的原理是什麼?它綜合了本書哪幾章的技術?

    提示與參考解答：原理：估計相鄰影格因抖動造成的整體運動，再反向補償、拉回穩定位置，最後裁掉黑邊。綜合：第 6 章相位相關法或第 12 章光流或第 13 章特徵匹配(估計運動)、第 4 章幾何轉換(反向補償)、第 12 章視訊框架。是多章技術的整合應用。

12. (綜合題)請設計一個「道路車速偵測」系統，說明如何從影片算出每輛車的實際速度(km/h)，並指出用到本書哪幾章的技術。

    提示與參考解答：流程：一、偵測車輛(背景相減或第 16 章 YOLO);二、追蹤每輛車的軌跡並維持 ID(第 12 章);三、量測車在連續影格間的像素位移;四、用第 4 章相機校正得到「每像素對應的實際距離(公尺)」把像素位移換算成實際位移;五、除以時間(位移影格數除以影格率得秒數)得速度，再換算 km/h。用到第 4 章(校正/像素換算)、第 12 章(偵測、追蹤、影格率提供時間基準)、第 16 章(YOLO 偵測)。

13. 把一段視訊沿著某一列切開，得到一張以 x 為橫軸、t 為縱軸的切片圖。靜止的背景與移動的物件在這張圖上各自長什麼樣子?若某條斜線在 100 格之內橫跨了 200 欄，該物件的速度是多少?

    提示與參考解答：靜止的背景在時間方向完全不變，畫出來是垂直的條紋;移動的物件則拉出一條斜線。斜率就是速度：200 欄 ÷ 100 格 = 2 像素 / 影格。圖 12-1 用這個方法量出來的速度與合成設定值完全吻合。這張切片圖值得記住，因為本章幾乎每一件事都是在讀這些斜線：移動偵測是找出哪裡不是垂直的，光流是量每一點的斜率，追蹤則是把同一條斜線從頭跟到尾。

14. 要說「相鄰影格幾乎相同」，該用什麼基準來比?若兩張影格各自帶著標準差 σ 的雜訊，相減之後雜訊的標準差是多少?再把單張影格與影格差分各自算熵，為什麼差分的熵低得多?這與 H.264、H.265 這類視訊編碼的設計有什麼關係?

    提示與參考解答：兩張獨立含雜訊的影格相減，雜訊的變異數相加，所以標準差變成 √2σ。取三倍即 3√2σ ≈ 4.24σ 作為「真的變了」的底線。σ=3 時這個底線是 12.7，而不是 3——若用 3 當判準，會把大量純粹的雜訊跳動當成真實變化。圖 12-2 實測在這條底線之下的像素佔 99.2%，剩下的幾乎都集中在移動物件的邊緣。一

般化的教訓：任何「有沒有變化」的判斷，門檻都必須從雜訊水準推出來，而不是寫死一個數字。至於資訊量：差值幾乎全部擠在零附近（分布極尖），而單張影格的灰階分布平坦得多，所以差分的熵低很多。圖 12-2 實測：單張影格 7.30 位元／像素，影格差分 4.19 位元／像素。用 JPEG 實際存一次，單張 17.6 KB、只存差分 5.9 KB，省下 67%。這正是視訊編碼的立足點：它們不逐張儲存影像，而是存少數關鍵影格（I 影格）加上大量的「與預測的差異」（P、B 影格）。注意反過來看的另一面——既然絕大部分內容都不變，那麼「變了的地方」就幾乎等於「有東西在動的地方」，這就是移動偵測的全部道理。壓縮與偵測，用的是同一個事實。

15. 30 fps 的視訊，每一影格的處理預算是多少毫秒?若某演算法在 320×240 上要 15 毫秒，換到 1920×1080 大約要多久?這對演算法選擇有什麼影響?

    提示與參考解答：30 fps 的預算是 1000/30 ≈ 33.3 毫秒，而且這還得包含讀取、解碼、顯示與寫出。耗時大致隨像素數成正比：(1920×1080)/(320×240) = 27 倍，所以約 15 × 27 ≈ 405 毫秒，遠超預算。圖 12-4 的實測顯示大部分前處理（灰階、模糊、CLAHE、Canny、型態學）都在 1 毫秒以內，但稠密光流是另一個量級。這解釋了為什麼實務上「稀疏光流 + 偶爾偵測」遠比「每一影格都算稠密光流」常見。實務上的對策還包括：縮小解析度處理、只在感興趣區域處理、或隔幾格才算一次。

16. MJPG 與 mp4v 這兩種編碼的根本差別是什麼?各自適合什麼場合?若要把影片當作法律證據保存，該選哪一種?

    提示與參考解答：MJPG 把每一影格各自存成 JPEG，完全不利用時間冗餘;mp4v 會做影格間預測，只記錄影格之間的差異。圖 12-5 實測同樣 90 張影格：MJPG 1582 KB、mp4v 744 KB，不到一半，而品質只差 1.5 dB。但 MJPG 的每一影格都能獨立解碼，隨機存取與逐格剪輯都方便，所以監控與工業相機常用它;mp4v 則必須從關鍵影格開始解。法律證據應選無失真編碼（如 FFV1）：圖 12-5 實測它仍能壓縮 4.6 倍，而且任何一個像素的誤差都是 0。注意「最大誤差」這個指標比平均 PSNR 更能看出失真的性格——這是第 11 章「失真的型態比失真的量更重要」那一課的延伸。

17. 設計一個實驗來決定影格差分的門檻。正確答案該怎麼定義才不會把「空心」的問題混進來?實測顯示最佳門檻與雜訊 σ 是什麼關係?

    提示與參考解答：關鍵在把兩件事分開。若用「完整的物件遮罩」當正確答案，量

到的其實是空心問題（召回率上限只有 6%），不是門檻的問題。正確做法是用同一組合成參數、不加雜訊再跑一次，以「乾淨序列中真的改變了的像素」當正確答案，這樣量到的才是門檻真正在做的事：把真實變化與雜訊分開。圖 12-7 這樣做之後，σ=3 時最佳門檻是 20，F1 達到 99.7%;把 σ 從 1 掃到 12，最佳門檻從 8 升到 63，擬合出約 4.9σ，與理論的 3√2σ ≈ 4.24σ 同一個量級。可操作的規則：取一塊靜止區域量出 σ，門檻設為約四倍。

18. 背景相減的輸出為什麼幾乎不可能直接使用?型態學的開運算與閉運算該用什麼順序、結構元素該多大?請說明結構元素過大時會發生什麼事。

提示與參考解答：輸出會布滿雜點、物件內部有破洞。順序是先開運算（侵蝕再膨脹）去掉比結構元素小的雜點，再閉運算（膨脹再侵蝕）補起內部破洞。圖 12-9 在 σ=14 的高雜訊下實測：未清理的 IoU 只有 0.410，開運算後 0.929，再閉運算 0.953。更具體的理由是連通元件數——未清理時多達 2725 個，而真實物件只有 3 個，也就是說若不清理就直接數物件，會得到荒謬的結果。但結構元素過大時 IoU 反而下降（邊長 13 時只剩 0.345），因為小的物件連同雜點一起被侵蝕掉了。結構元素的大小應由「最小的目標物件」決定，而不是由雜訊決定。

19. MOG2 的 detectShadows 是怎麼判斷陰影的?它的輸出有幾種值、程式裡若寫成 mask > 0 會發生什麼事?另外，MOG2 的背景更新大致是 B ← (1−α)·B + α·I，請說明模型的「記憶長度」約為多少影格，以及為什麼同一個 α 同時決定了「學多快」與「忘多快」。若要偵測「被遺留在月台上的包裹」，α 該怎麼設?

提示與參考解答：判斷依據是「陰影只讓像素變暗，但照到的還是同一塊地面，所以紋理與色度都沒變、只有亮度等比例下降」。因此檢查「目前像素值除以背景模型像素值」是否落在固定區間（例如 0.5~0.95）且色度沒有明顯偏移，就能標成陰影。輸出是三態：0 背景、255 前景、127 陰影。若寫成 mask > 0，127 會被當成前景，這個功能等於沒開。正確寫法是 mask == 255。圖 12-10 實測：關閉時 IoU 0.847、陰影區有 98% 被誤判;開啟並正確濾掉 127 之後 IoU 升到 0.962、陰影誤判降到 1%。詳見 12.3.3 節。至於學習率：把遞迴式展開，n 格之前的影格貢獻 $\alpha(1-\alpha)^n$ 的權重，所以記憶長度約為 1/α 格：α=0.01 記得住百來格，α=0.05 只記得住二十格左右。學得快與忘得快是同一個 α 的兩面，無法分開調。圖 12-11 實測：α=0.05 時暖身只要 6 格，但物件停住後 2 格就被吃掉;α=0.001 時物件可以撐

102 格，代價是背景模型到第 170 格都還沒學乾淨。遺留物偵測需要極小的 α，而更好的做法是同時維護一個長期與一個短期模型，取「短期認為是背景、長期認為是前景」的區域——那就是「最近才出現並停住」的東西。

20. 光流的亮度恆定假設被破壞時，演算法會報錯嗎?乘性亮度變化（如光圈改變）與加性亮度變化（如曝光補償）哪一個傷害比較大?請從光流方程式解釋，並說明實務上的對策。

提示與參考解答：不會報錯，只會安靜地給出錯誤答案——這比報錯更危險。圖 12-12 實測顯示加性變化的傷害遠大於乘性：亮度乘以 1.6 時誤差只有 0.70 像素，但整體加 60 階時誤差高達 21.85 像素。從 $I_x \cdot u + I_y \cdot v + I_t = 0$ 看：整體加一個常數，空間梯度 I_x、I_y 完全不變，卻在每一點都塞進一個假的時間變化 I_t，解出來的速度因此被整個推歪;乘性變化則同時放大了空間梯度與時間變化，兩者在相除時大部分抵消掉了。對策：關掉相機的自動曝光與自動白平衡，或在算光流之前先做亮度正規化（第 5 章的直方圖等化或 CLAHE）。

21. 請寫出光流方程式，並用它說明孔徑問題為什麼不是「演算法不夠好」。Lucas-Kanade 用什麼假設補上缺少的那條方程式，這個補救與第 13 章的 Harris 角點有什麼關係?又，為什麼把窗口從 15 放大到 31 幾乎無助於追蹤更大的位移，金字塔卻有效?

提示與參考解答：由亮度恆定假設做一階泰勒展開得 $I_x \cdot u + I_y \cdot v + I_t = 0$——一條方程式、兩個未知數，所以單一像素只能定出梯度方向（邊緣法線）的分量，沿邊緣的分量完全沒有被約束。這是資訊本身不足，不是演算法的缺陷。圖 12-13 的實測：改變窗口大小得到四組完全不同的位移，但投影到條紋法線方向後全部一致，因為這些解產生的畫面完全相同。Lucas-Kanade 補上的假設是「小窗口內所有像素運動相同」，於是 N 個像素給 N 條方程式，用最小平方法解一個 2×2 方程組。那個 2×2 矩陣正是 Harris 用的結構張量，它可逆的條件就是「兩個特徵值都夠大」，也就是角點的定義。所以「挑角點來追蹤」不是經驗法則，而是方程組有唯一解的必要條件。詳見 12.4.4 節。至於金字塔：限制來自泰勒展開的一階近似，它的有效範圍由影像本身的紋理尺度決定，把窗口開大只是納入更多同樣失效的方程式。圖 12-14 實測：窗口 15 與窗口 31 的可追蹤位移都是 5 像素，完全沒有改善。金字塔的做法是先把兩張影格縮小 2^n 倍，在最粗的那一層上原本 80 像素的位移看起來只剩幾個像素，線性近似重新成立;估出粗略位移後放大一倍當作下一層的起點，逐層

只需估一個小修正量，等於把「大位移」化約成一連串「小位移」。實測：加上 3 層金字塔後可追位移從 5 跳到 87 像素，5 層超過 160 像素。這正是第 11 章多解析度分析的再一次勝利。實務推論：目標移動很快時，與其調大窗口，不如增加 maxLevel 或提高影格率。

22. 稠密光流的運動場常用 HSV 視覺化。請說明色相與亮度各編碼什麼，並描述「相機往左平移」「繞光軸旋轉」「往前推進」三種情況各會呈現什麼圖樣。什麼是擴張焦點?它有什麼用?

    提示與參考解答：色相編碼運動方向、亮度編碼運動大小（第 8 章的 HSV）。往左平移：整張圖是單一顏色（所有像素同方向同速）。繞光軸旋轉：顏色繞著中心轉一圈，中心最暗（轉軸上不動）、邊角最亮（半徑最大、線速度最快）。往前推進：顏色從中心向外放射，中心那個不動的點就是擴張焦點（Focus of Expansion），它指出相機正在往哪裡前進。擴張焦點是視覺里程計與障礙閃避的關鍵線索，也是 12.7.4 節機器人案例的核心。一般化的讀法：均勻色塊代表平移、漩渦代表旋轉、放射狀代表前進或後退。

23. 追蹤器為什麼一定會漂移?若漂移速率是每格 d 像素、可容忍偏移是 D 像素，偵測的校正週期 k 該怎麼估?請說明這個旋鈕連接了哪兩件事。

    提示與參考解答：追蹤的每一格都以上一格為起點，所以每格引入的誤差不會被修正，只會被帶進下一格累積。若誤差是獨立隨機的，n 格後約累積 $\sigma\sqrt{n}$;若有系統性偏向（例如樣板每格都多學了一點背景），則是 $n\cdot\sigma$ 的線性累積。校正週期應滿足 $k \le D/d$。圖 12-17 實測：純追蹤第 39 格就跌破 IoU 0.5，末格只剩 0.30;每 30 格校正一次平均 0.79，每 10 格校正一次 0.92。成本面：平均每格運算量約為「偵測成本 ÷ k + 追蹤成本」。所以 k 把準確度與算力接在一起——這也是圖 12-4 的耗時實測要放在前面的原因。詳見 12.5.3 節。

24. 追蹤器在遮擋時失敗的典型路徑是什麼?為什麼說問題不在「找不到」而在「找錯了還學起來」?請提出一個最小的補救方案並說明它的極限。

    提示與參考解答：遮擋時比對分數最高的位置往往落在遮擋物上，若追蹤器照例把那裡更新成新樣板，它就改追遮擋物了，遮擋結束後再也接不回來。圖 12-18 實測：純樣板追蹤在遮擋結束後的平均 IoU 只剩 0.09。最小補救是設兩道門檻——分數夠高才相信位置，分數非常高才更新樣板;兩者都不滿足時不更新任何東西，改用

> *先前估到的速度外推，靠慣性撐過遮擋。實測這讓遮擋後的平均 IoU 從 0.09 回到 0.57，約七倍。極限是：外推 25 格之後速度估計的誤差已經累積了好幾個像素，所以還沒完全對準。實務上要再補兩件事——卡爾曼濾波器（維護預測的不確定度，遮擋愈久搜尋範圍愈寬）與外觀再識別向量（DeepSORT 的 re-ID，確認重新出現的是不是原來那一個）。*

25. (綜合題)為什麼跌倒偵測不能只看單張影像?請設計一個最簡單的時序指標來分辨「跌倒」與「慢慢躺下」，說明它會在什麼情況下誤判，以及照護場景還要補上什麼考量。

> *提示與參考解答：兩者的最後一格完全相同——都是「人躺在地上」，任何只看單張影像的分類器在這裡都只能猜。分辨的資訊不在任何一張影格裡，而在影格之間。最簡單的指標是影格間變化量（或身體主軸角度的變化率）：圖 12-20 實測跌倒的變化率尖峰是 10.6，慢慢躺下只有 2.5，差了 4.3 倍。誤判情況：「快速坐下」「彎腰撿東西」「突然低頭」都可能產生高變化率。所以真實系統還要加上後續條件，例如「倒地後是否長時間不動」、身體與地面的角度、以及事後的姿態維持時間。照護場景的額外考量：誤報會頻繁打擾長者、漏報可能延誤救援，兩者後果都嚴重;且涉及被監控者的隱私與尊嚴，應審慎設計（例如只在本地運算、不上傳影像、或改用不成像的感測器）。*

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018.(相關章節)
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 9(視訊處理與運動)。
- Muybridge, E., The Horse in Motion (Sallie Gardner at a Gallop), 1878, Library of Congress.(本章叫獸開講之史料來源)
- Lucas, B. D., & Kanade, T., An Iterative Image Registration Technique with an Application to Stereo Vision, IJCAI, 1981.

- Farnebäck, G., Two-Frame Motion Estimation Based on Polynomial Expansion, SCIA, 2003.
- Bewley, A., et al., Simple Online and Realtime Tracking (SORT), ICIP, 2016;Wojke, N., et al., DeepSORT, ICIP, 2017.

- 補充教材與課程資訊：@prof(https://page.line.me/prof)